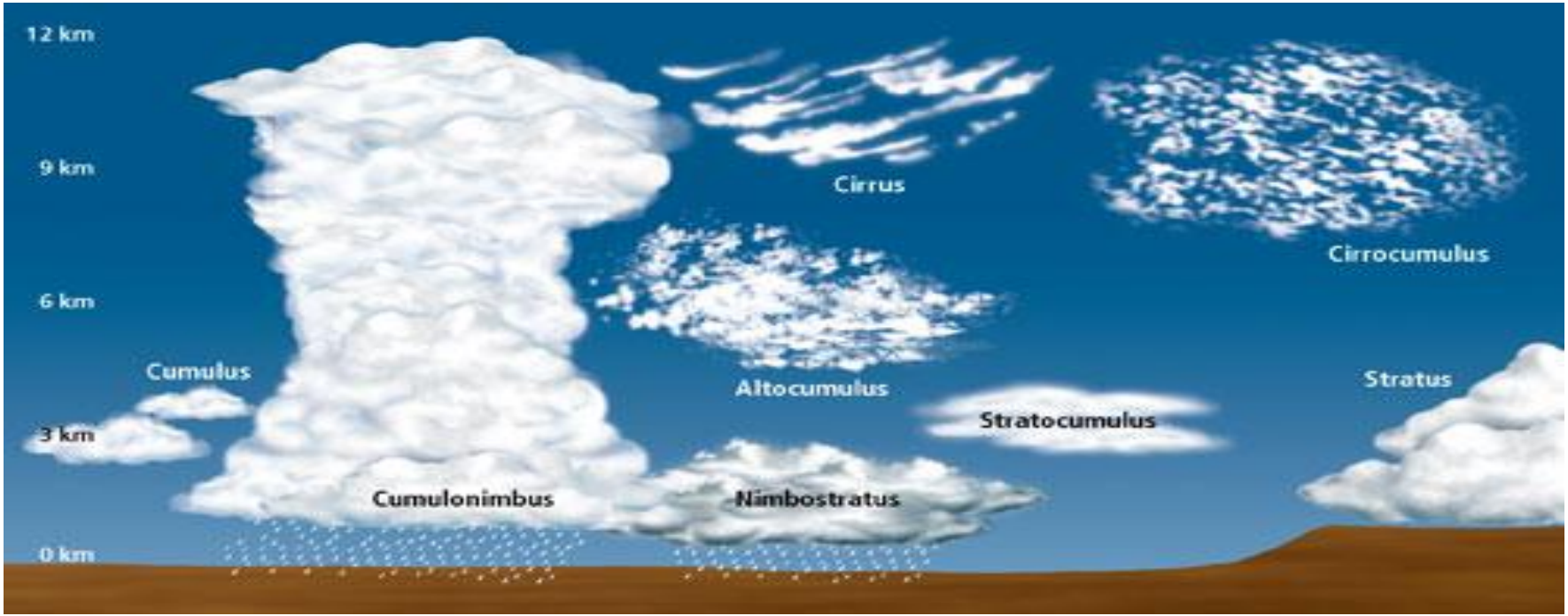
Definition of Cloud Computing

Cloud Classification



STANDARD CLOUD TYPE CHART

Cirrus (above)

Cirrostratus (above)

Cirrocumulus (above)

Altostratus (above)

Altocumulus (above)

Stratus (above)

Stratocumulus (above)

Nimbostratus (above)

Cumulus (above)

Cumulus congestus (above)

Cumulonimbus (above)

Wall cloud (above)

Shelf cloud (above)

Fractus (scud) (above)

Mammatus (above)

Contrails (above)

Fog (above)

Cloud chart showing the different types of high, mid, and low-level clouds, as well as a number of other interesting cloud types and formations.

NIST Definition of Cloud Computing

- NIST(National Institute of Standards and Technology) Special Publication 800-145http://csrc.nist.gov/publications/nistpubs/800-145/SP800-145.pdf
- Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.

- This cloud model is composed of **five** essential characteristics,

Five Essential Characteristics

1. **On-demand self-service**.

   A consumer can unilaterally provision computing capabilities, such as server time and network storage, as needed automatically without requiring human interaction with each service provider

Five Essential Characteristics

2. **Broad network access**.

   Capabilities are available over the network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, tablets, laptops, and workstations).

Five Essential Characteristics

3. **Resource pooling**.

The provider's computing resources are pooled to serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand. There is a sense of location independence in that the customer generally has no control or knowledge over the exact location of the provided resources but may be able to specify location at a higher level of abstraction (e.g., country, state, or datacenter). Examples of resources

Five Essential Characteristics

4. **Rapid elasticity**. Capabilities can be elastically provisioned and released, in some cases automatically, to scale rapidly outward and inward commensurate with demand. To the consumer, the capabilities available for provisioning often appear to be unlimited and can be appropriated in any quantity at any time.

Five Essential Characteristics

5. **Measured Service**.
   Cloud systems automatically control and optimize resource use by leveraging a metering capability* at some level of abstraction appropriate to the type of service (e.g., storage, processing, bandwidth, and active user accounts). Resource usage can be monitored, controlled, and reported, providing transparency for both the provider and consumer of the utilized service.

*Typically this is done on a pay-per-use or charge-per-use basis.

Three Service Models

1. **Cloud Software as a Service (SaaS)**

The capability provided to the consumer is to use the provider's applications running on a cloud infrastructure*. The applications are accessible from various client devices through either a thin client interface, such as a web browser (e.g., web-based email), or a program interface. The consumer does not manage or control the underlying cloud infrastructure including network, servers, operating systems, storage, or even individual application capabilities, with the possible exception of limited user-specific application configuration settings

Three Service Models

2. Cloud Platform as a Service (PaaS).

The capability provided to the consumer is to deploy onto the cloud infrastructure consumer-created or acquired applications created using programming languages, libraries, services, and tools supported by the provider*. The consumer does not manage or control the underlying cloud infrastructure including network, servers, operating systems, or storage, but has control over the deployed applications and possibly configuration settings for the application-hosting environment.

*This capability does not necessarily preclude the use of compatible programming languages, libraries, services, and tools from other sources.

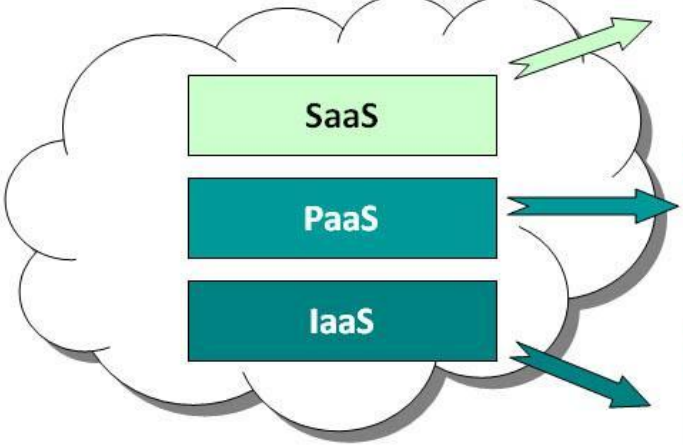
Three Service Models

3. Cloud Infrastructure as a Service (IaaS).

The capability provided to the consumer is to provision processing, storage, networks, and other fundamental computing resources where the consumer is able to deploy and run arbitrary software, which can include operating systems and applications. The consumer does not manage or control the underlying cloud infrastructure but has control over operating systems, storage, and deployed applications; and possibly limited control of select networking components

Cloud Computing - Services

Software as a Service - SaaS
Platform as a Service - PaaS
Infrastructure as a Service - IaaS



Who Uses It	What Services are available	Why use it?
Business Users	EMail, Office Automation, CRM, Website Testing, Wiki, Blog, Virtual Desktop ...	To complete business tasks
Developers and Deployers	Service and application test, development, integration and deployment	Create or deploy applications and services for users
System Managers	Virtual machines, operating systems, message queues, networks, storage, CPU, memory, backup services	Create platforms for service and application test, development, integration and deployment

Four Development Models

1. **Private cloud**.

   The cloud infrastructure is provisioned for exclusive use by a single organization comprising multiple consumers (e.g., business units). It may be owned, managed, and operated by the organization, a third party, or some combination of them, and it may exist on or off premises.

Four Development Models

2. **Community cloud**.
   The cloud infrastructure is provisioned for exclusive use by a specific community of consumers from organizations that have shared concerns (e.g., mission, security requirements, policy, and compliance considerations). It may be owned, managed, and operated by one or more of the organizations in the community, a third party, or some combination of them, and it may exist on or off premises.

Four Development Models

3. **Public cloud**.

The cloud infrastructure is provisioned for open use by the general public. It may be owned, managed, and operated by a business, academic, or government organization, or some combination of them. It exists on the premises of the cloud provider.

Four Development Models

4. **Hybrid cloud**.
   The cloud infrastructure is a composition of two or more distinct cloud infrastructures (private, community, or public) that remain unique entities, but are bound together by standardized or proprietary technology that enables data and application portability (e.g., cloud bursting for load balancing between clouds).

Summary

- Five essential characteristics: On-demand self-service; Broad network access; Resource pooling; Rapid elasticity; and Measured Service.
- Three service models: Software as a Service (SaaS); Platform as a Service (PaaS); and Infrastructure as a Service (IaaS).
- Four deployment models: Private cloud; Community cloud; Public cloud; and Hybrid cloud.
- Is this definition sufficient?

What you need to do this week?

- Resource can be found at Canvas
  - Read “How to Search Reference Papers”, search and download 5 papers that you have interest in Cloud Computing & Security.
  - Read the paper “Intel guide of how to build a cloud system” and get familiar with cloud virtualization system.
  - Get to know each other and compose group partners
    - Use Canvas’s discussion board to share your thoughts and search for project partners
  - Investigate project topics

Virtualization Technologies

CPSC 454 Cloud Computing & Security

Portions of this PPT draw from PPT authored by Professor Dijiang Huang at Arizona State University

Outline

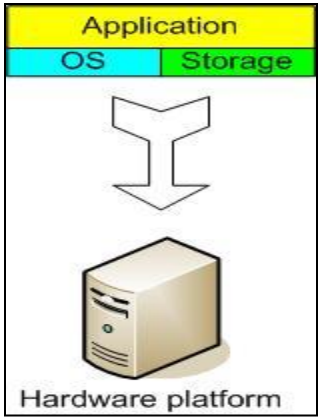
- Virtualization Concepts and Classifications
- I/O Virtualization
- VMM (Virtual Machine Monitor)

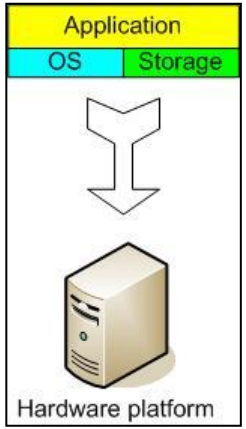
Virtualization Concepts and Classification

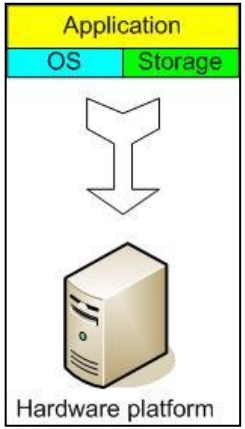
Virtualization: What is it, really?

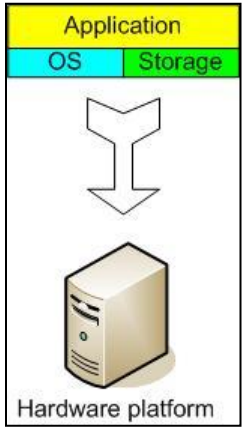
- Real vs. Virtual
  - Similar essence, effect
  - *"Formally"* different
- A framework that combines or divides [computing] resources to present a *transparent* view of one or more environments
  - Hardware/software partitioning (or aggregation)
  - Partial or complete machine simulation
  - Emulation (again, can be partial or complete)
  - Time-sharing (in fact, sharing in general)
  - In general, can be M-to-N mapping (M "real" resources, N "virtual" resources)
    - Examples: VM (M,N), Grid Computing (M,1), Multitasking (1,N)

The Traditional Server Concept



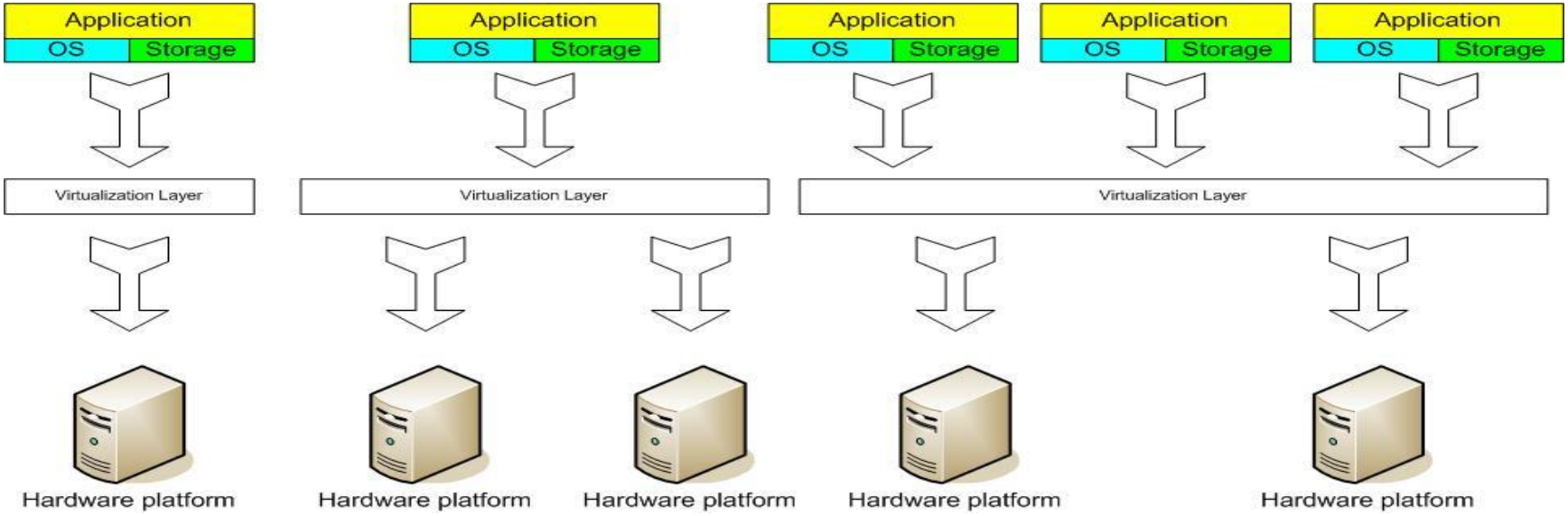






Web Server	App Server	DB Server	EMail
Windows	Linux	Linux	Windows
IIS	Glassfish	MySQL	Exchange

The Virtual Server Concept



Virtual Machine Monitor (VMM) layer between *Guest OS* and hardware

Virtualization Products

Implementation	Virtualization Type	Installation Type	License
Bochs	Emulation	Hosted	LGPL
QEMU	Emulation	Hosted	LGPL/GPL
VMware	Full Virtualization & Paravirtualization	Hosted and bare-metal	Proprietary
User Mode Linux(UML)	Paravirtualization	Hosted	GPL
Lguest	Paravirtualization	Bare-metal	
OpenVZ	OS level	Bare-metal	GPL
Linux VServer	OS level	Bare-metal	GPL
Xen	Paravirtualization or Full when using hardware extensions	Bare-metal	GPL
Parallels	Full Virtualization	Hosted	Proprietary
Microsoft	Full Virtualization	Hosted	Proprietary
z/VM	Full Virtualization	Hosted and bare-metal	Proprietary
KVM	Full Virtualization	Bare-metal	GPL
Solaris Containers	OS level	Hosted	CDDL
BSD Jails	OS level	Hosted	BSD
Mono	Application Layer	Application Layer	Compiler and tools GPL, Runtime libraries LGPL, Class libraries MIT X11
Java Virtual Machine	Application Layer	Application Layer	GPL

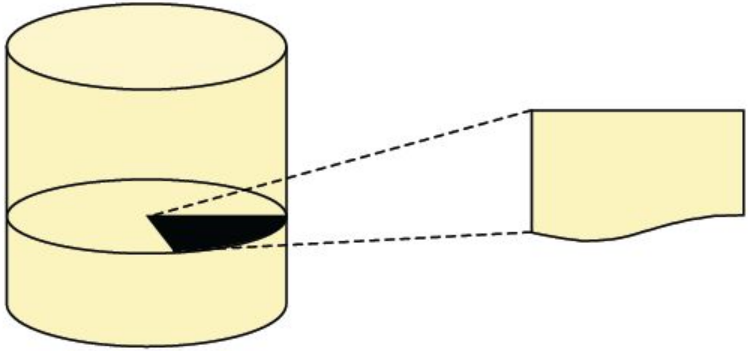
Virtualization: Why?

- Server consolidation
- Application Consolidation
- Sandboxing
- Multiple execution environments
- Virtual hardware
- Debugging
- Software migration (Mobility)
- Appliance (software)
- Testing/Quality Assurance

Virtualization

- A layer mapping its visible interface and resources onto the interface and resources of the underlying layer or system on which it is implemented
- Purposes
  - Abstraction – to simplify the use of the underlying resource (e.g., by removing details of the resource's structure)
  - Replication – to create multiple instances of the resource (e.g., to simplify management or allocation)
  - Isolation – to separate the uses which clients make of the underlying resources (e.g., to improve security)

Abstraction



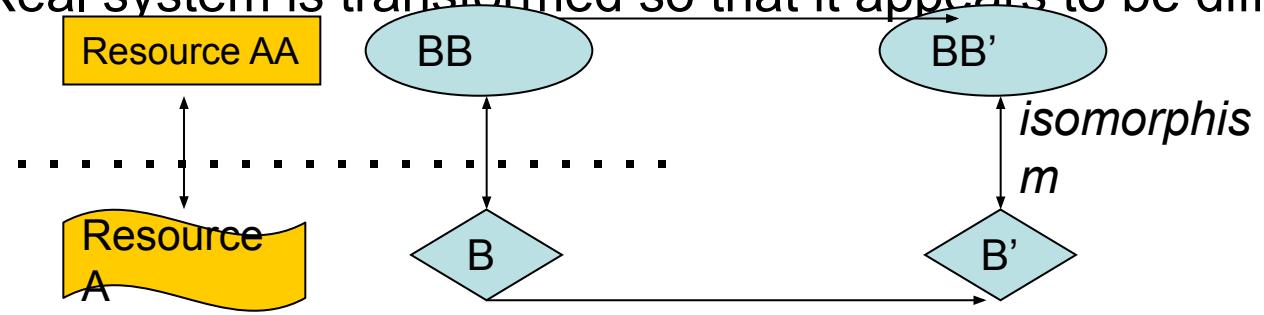
- Abstraction is about hiding details
  - a file on a hard disk
    - mapped to a collection of sectors and tracks on the disk
    - we don't directly address disk layout when accessing the file
  - a level of abstraction provides a simplified interface to underlying resources

Abstraction

- **Why Abstraction?**
  - it is a key to managing complexity in computer systems
    - different abstraction levels: hierarchy + interfaces
  - interfaces simplify your life!
    - no need to deal with too many details
    - create a certain level of vendor independence
- **Abstraction Limitations**
  - diversity reduces interoperability
  - instruction sets, operating systems, programming languages
  - operating system designs reduce flexibility
    - operating systems introduce abstractions for memory, I/O, ...
    - typical approach: the OS manages resources directly
    - implicit assumption: all resources under a single regime

Abstraction, Virtualization of Computer System

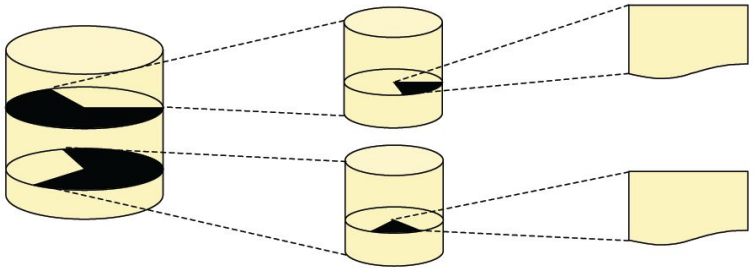
- Virtualization
  - Similar to Abstraction but it does not always hide low layer's details
  - Real system is transformed so that it appears to be different



  - Virtualization can be applied not only to subsystem, but to an

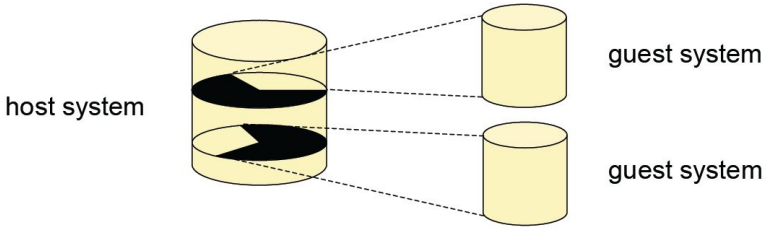
Virtualization

- Virtualization is about creating illusions



  - two files on separate hard disks, each of which is a partition on an actual hard disk
  - virtualization:
    - a virtualized system's interface and resources are mapped onto interface and resources of another ("real") system
    - virtualization provides a different interface and/or resources at the same level of abstraction

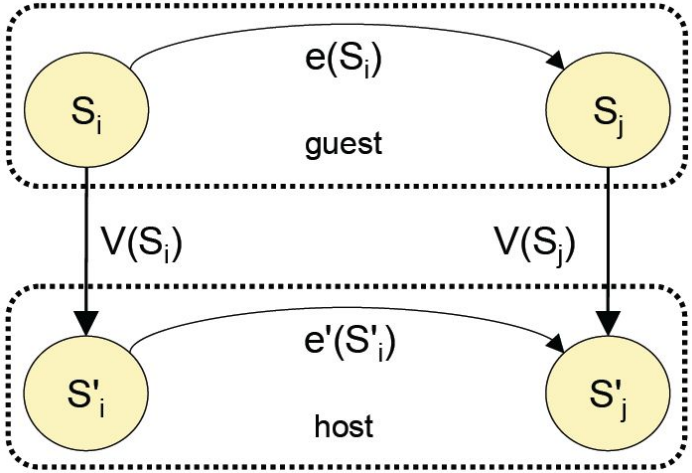
Virtualization



- two types of systems involved
  - the virtualized system is called guest system
  - the "real" system is called host system
- multiple layers of virtualization
  - virtualized host system is also an option

Virtualization

- the host is transformed
  - appears to be a (set of) different *virtualized* system(s)
  - introduction of an isomorphism mapping guest to host system



Si, Sj: states in a system

e: operation sequence modifying Si to Sj

V: function mapping guest states to host states

S'i, S'j: host states

e': operation sequence corresponding to e

- virtualization vs. abstraction
  - the same isomorphism can also be used to depict abstraction
  - important difference: virtualization does not necessarily hide details

What Can be Virtualized

- Computers
- Storage
- Network
- Services
- Security
- … almost everything
- What we see the mostly is to virtualize a computer.

Virtual Machine

- **"Virtual"**
  - what we know now
    - virtualization is about creating illusions
    - map interfaces and resources to other interfaces and resources
- **"Machine"?**
  - what we need to know: notion of "machine"
    - what is virtualized to build a virtual machine?
    - what interfaces and resources are there that can be subject to
- **virtualization?**
  - roadmap
    - look at computer system architecture
    - identify (types of) interfaces
    - identify and characterize virtualization approaches

Virtualization is “HOT”

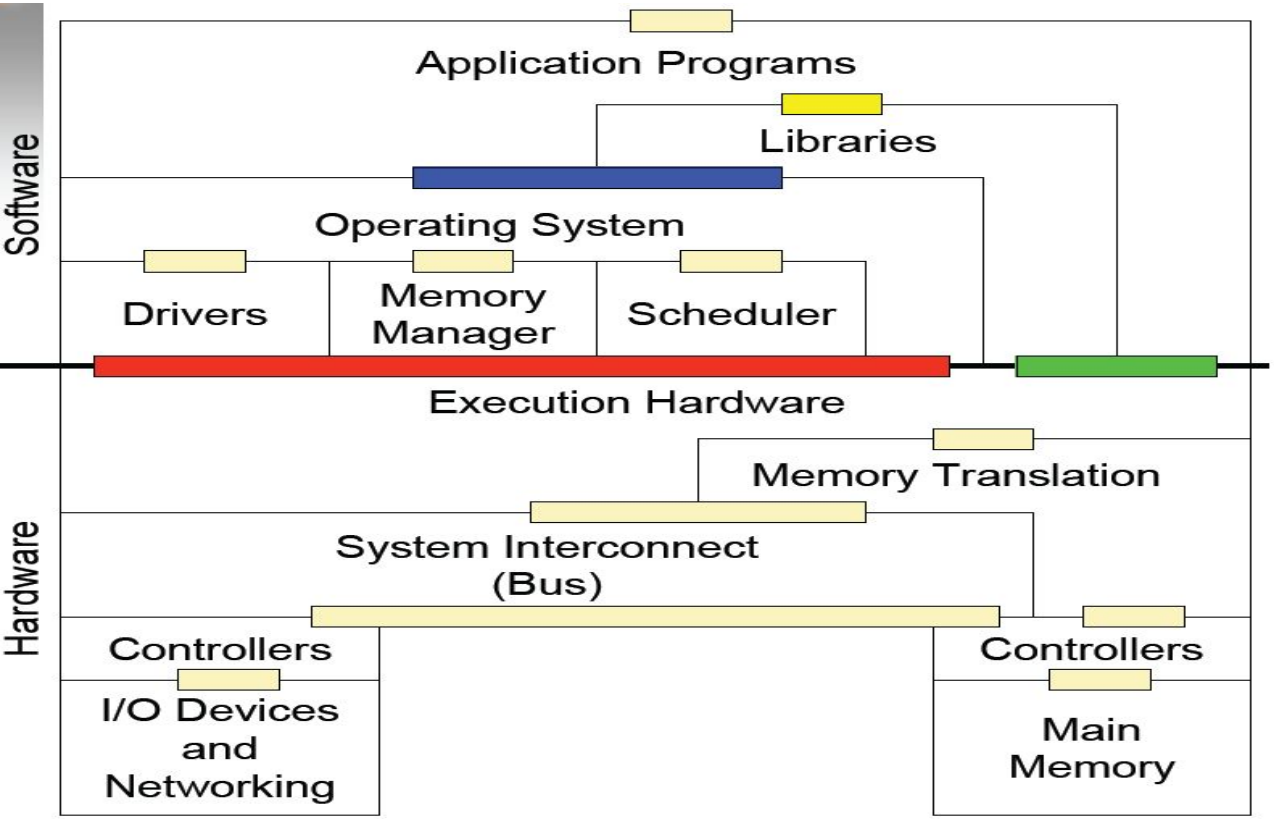
- Microsoft acquires Connectix Corp.
- EMC acquires VMware
- Veritas acquires Ejascent
- IBM, already a pioneer
- Sun working hard on it
- HP picking up

Virtualization is HOT!!!

First Virtualization Classification

- ISA based virtualization
- ABI based virtualization
- API based virtualization

Computer Architecture



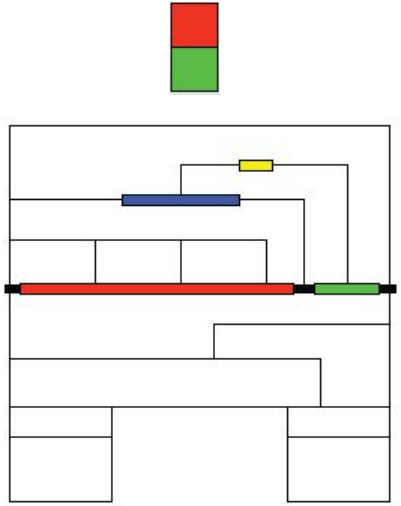
Distinct Interfaces

Instruction Set Architecture (ISA)

Application Binary Interface (ABI)

Application Programming Interface (API)

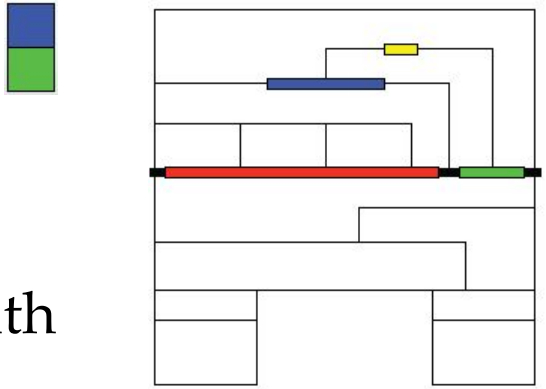
Instruction Set Architecture



- division between hardware and software
- sub-parts
  - user ISA parts of the ISA visible to applications
  - system ISA parts of the ISA visible to supervisor software
  - system ISA can also employ user ISA components
- software compatibility
  - software built to a given ISA can run on any

Application Binary Interface (ABI)

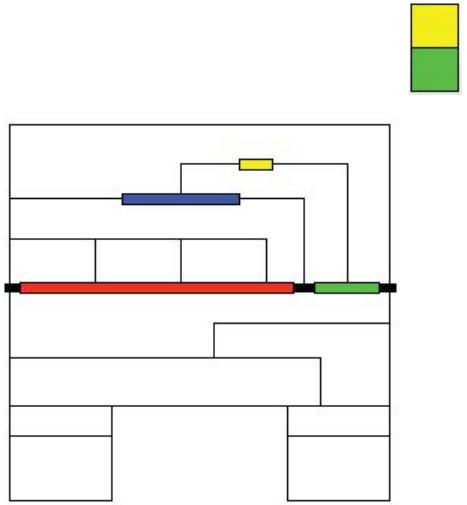
- provides programs with access to hardware resources and services
- major components
  - set of all user instructions
  - system calls : indirect interaction with hardware resources



- system calls
  - OS operations performed on behalf of user programs
  - often includes security checks (wrt. access privileges)
- support for portability
  - binaries compiled to a specific ABI can run unchanged on a system with the same ISA and OS

Application Programming Interface (API)

- abstracts from the details of service implementations
- usually defined with respect to a high-level language (HLL)
  - standard library to invoke OS services
  - typically defined at source code level
- support for portability
  - software using a given API can be ported to other platforms by recompilation



"Machine": Matter of Perspective

Process Perspective

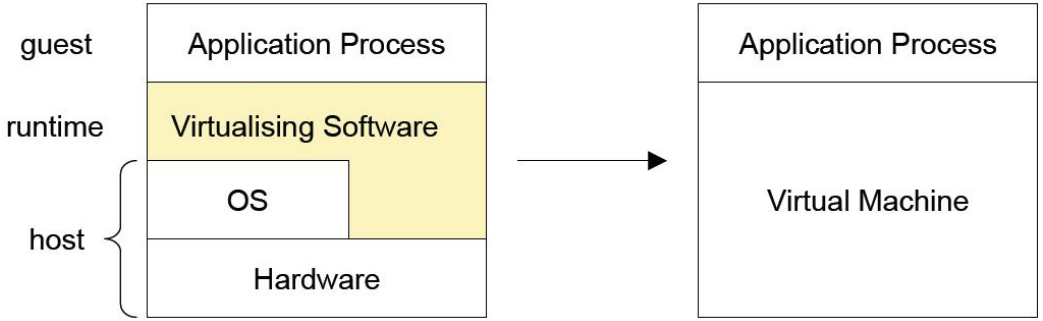
- machine = combination of OS and user-level
  - logical memory address space assigned to the process
  - user-level registers and instructions for program execution
  - I/O part of machine visible only through OS; interaction via OS calls
- ABI provides process/machine interface

Operating System Perspective

- machine is implemented by underlying alone
  - system: full execution environment for multiple users
  - processes share file system and other resources
  - OS is part of the system
- ISA provides system/machine interface

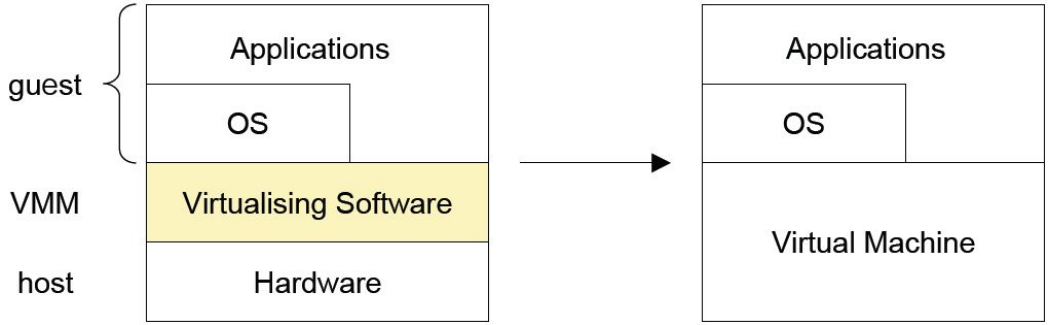
Process Virtual Machines

- capable of supporting an individual process
- virtualizing software
  - placed at ABI, on top of OS + hardware
  - emulates both user-level instructions and OS calls



System Virtual Machines

- provide a complete system environment
  - can support a "guest OS" with (probably) many user processes
- virtualizing software ("virtual machine monitor", VMM)
  - placed between underlying hardware and conventional software
  - ISA translation
  - alternative approach: hosted VM (virtualizing software built *on tc*



Second Virtualization Classification

- Instruction Set Architecture
  - Emulate the ISA in software
    - Interprets, translates to host ISA (if required)
    - Device abstractions implemented in software
    - Inefficient
  - Optimizations: Caching, Code reorganization.
  - Applications: Debugging, Teaching, multiple OS

- Hardware Abstraction Layer (HAL)
  - Between "real machine" and "emulator" (maps to real hardware)
  - Handling non-virtualizable architectures
  - Applications: Fast and usable, virtual hardware, consolidation, migration

Second Virtualization Classification cont'd

- Operating System Level
  - Virtualized SysCall Interface
  - May or may not provide all the device abstractions
  - Easy to manipulate (create, configure, destroy)
- Library (user-level API) Level
  - Presents a different subsystem API to application
  - Complex implementation, if kernel API is limited
  - User-level device drivers
- Application (Programming Language) Level
  - Virtual architecture (ISA, registers, memory, …)
  - Platform-independence (□ highly portable)
  - Less control on the system (extremely high-level)

Overall Picture

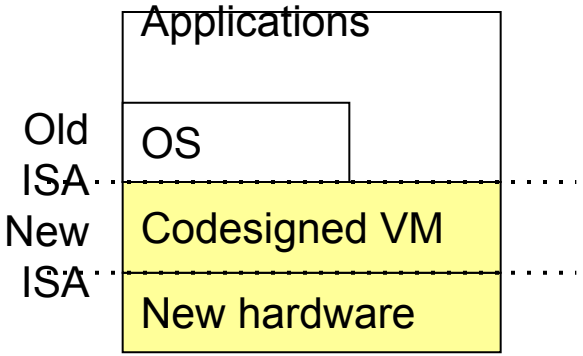
	ISA	HAL	OS	Library	PL
Performance	*	****	****	***	**
Flexibility	****	***	**	**	**
Ease of Impl	**	*	***	**	**
Degree of Isolation	***	****	**	**	***

(more stars are better)

Instruction Set Architecture Level Virtualization

- Technologies
  - Emulation: Translates guest ISA to native ISA
  - Emulates h/w specific IN/OUT instructions to mimic a device
  - Translation Cache: Optimizes emulation by making use of similar recent instructions
  - Code rearrangement
  - Speculative scheduling (alias hardware)
- Issues
  - Efficient Exception handling
  - Self-modifying code

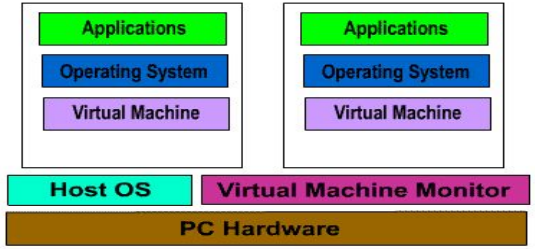
ISA Level Virtualization: Examples



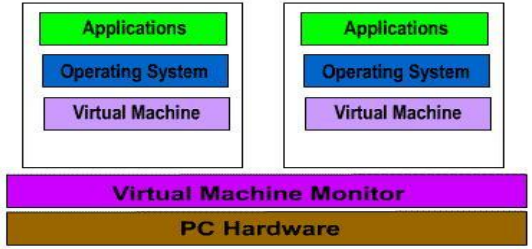
- Bochs: Open source x86 emulator
  - Emulates whole PC environment
    - x86 processor and most of the hardware (VGA, disk, keyboard, mouse, …)
    - Custom BIOS, emulation of power-up, reboot
    - Host ISAs: x86, PowerPC, Alpha, Sun, and MIPS
- Crusoe (Transmeta)
  - "Code morphing engine" – dynamic x86 emulator on VLIW processor
  - 16 MB "translation cache"
  - Shadow registers: Enables easy exception handling
- QEMU:
  - Full Implementation
    - Multiple target ISAs: x86, ARM, PowerPC, Sparc
    - Supports self-modifying code
    - Full-software and simulated (using mmap()) MMU

HAL Virtualization Techniques

- Standalone vs. Hosted
  - Drivers
  - Host and VMM worlds
  - I/O
- Protection Rings
  - Multilevel privilege domains
- Handling "silent" fails
  - Scan code and insert/replace artificial traps

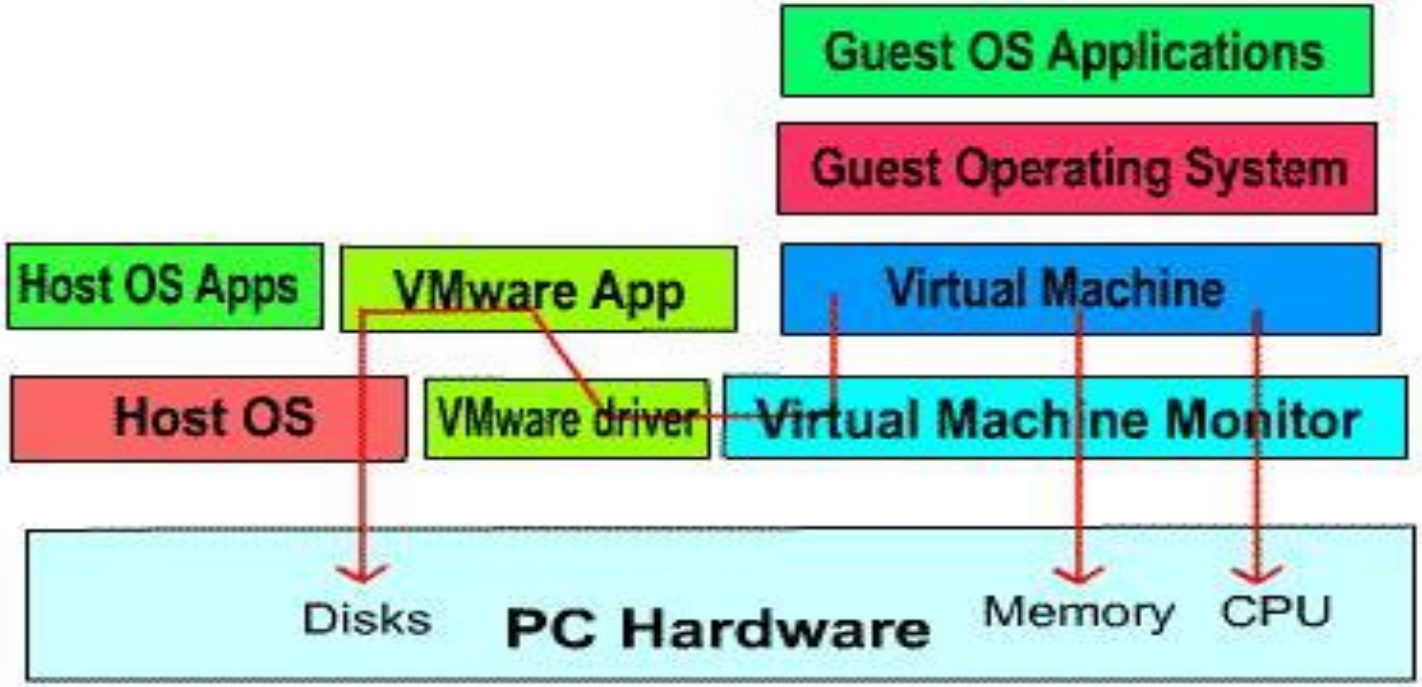


Hosted Virtual Machine



Stand Alone Virtual Machine

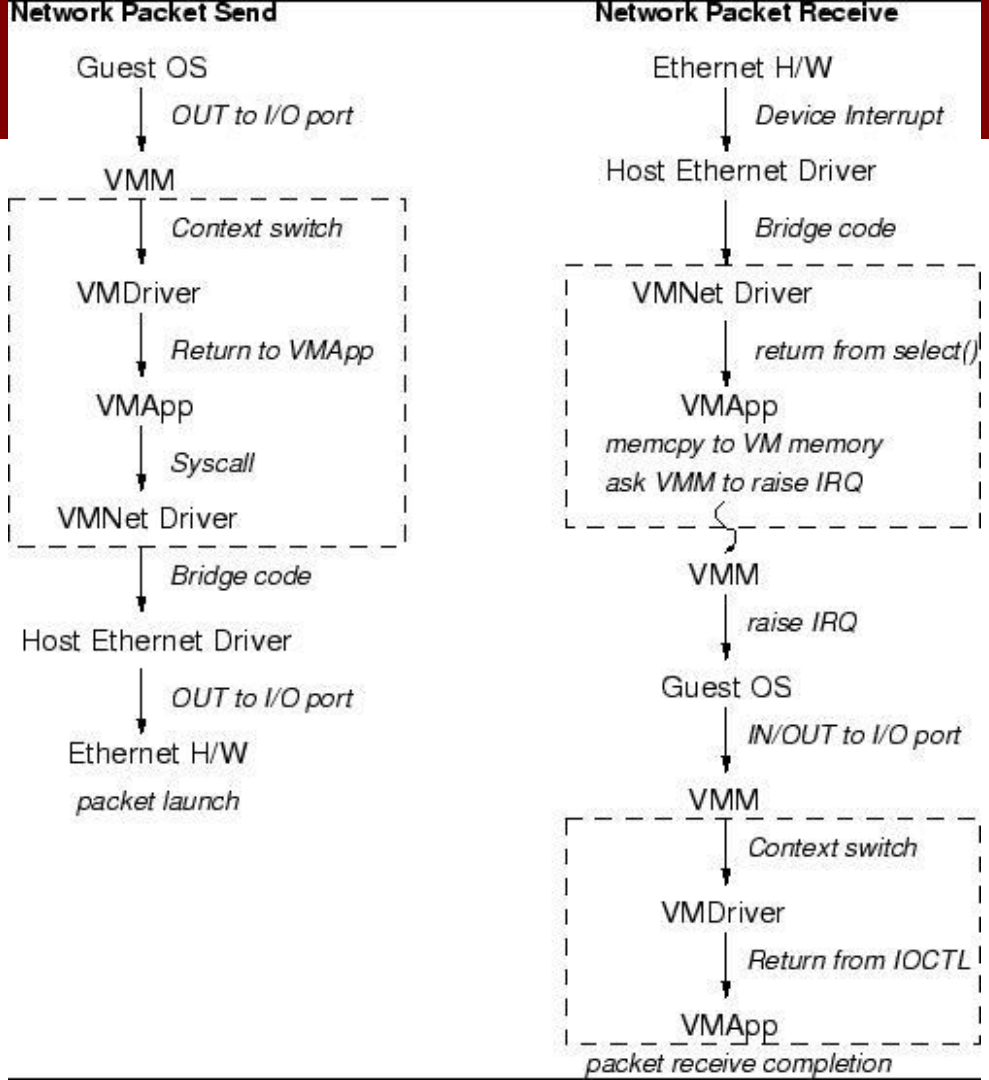
VMware Architecture



VMware Workstation Architecture

VMware: I/O Virtualization

- VMM does not have access to I/O
- I/O in "host world"
  - Low level I/O instructions (issued by guest OS) are merged to high-level I/O system calls
  - VM Application executes I/O SysCalls
- VM Driver works as the communication link between VMM and VM Application
- World switch needs to "save" and "restore" machine state
- Additional techniques to increase efficiency


Network Packet Send
Guest OS
OUT to I/O port
VMM
Context switch
VMDriver
Return to VMApp
VMApp
Syscall
VMNet Driver
Bridge code
Host Ethernet Driver
OUT to I/O port
Ethernet H/W
packet launch
Network Packet Receive
Ethernet H/W
Device Interrupt
Host Ethernet Driver
Bridge code
VMNet Driver
return from select()
VMApp
memcpy to VM memory
ask VMM to raise IRQ
VMM
raise IRQ
Guest OS
IN/OUT to I/O port
VMM
Context switch
VMDriver
Return from IOCTL
VMApp
packet receive completion

Paravirtualization

- Traditional architectures do not scale
  - Interrupt handling
  - Memory management
  - World switching
- Virtualized architecture interface
  - Much simpler architectural interface
  - Virtual I/O and CPU instructions, registers, …
- Portability is lost

Paravirtualizing the Memory Management Unit (MMU)

- Guest OS allocate and manage own page-tables
  - Hypercalls to change PageTable base.
- Xen Hypervisor is responsible for trapping accesses to the virtual page table, validating updates and propagating changes.
- Xen must validate page table updates before use
  - Updates may be queued and batch processed
- Validation rules applied to each PTE
  - Guest may only map pages it owns
- XenoLinux implements a balloon driver
  - Adjust a domain's memory usage by passing memory pages back and forth between Xen and XenoLinux

Examples

- Denali
  - Simpler customized OS with no VM for network applications
- Xen
  - Simpler port to commercial OS
  - Exposes some “real” hardware, e.g. clock, physical memory address

OS Level Virtualization

- Containers (operating environments) on top of OS
  - Processes, File System, Network resource (IP address), Environment variables, System call interface
- Technologies
  - chroot(): File system virtualization on Unix
  - Name spaces: Each container is tagged and new entities (fork()) generated from a container remains inside
  - System call interposition: The only interface with user space, can modify parameters, return values (to expose a different environment)
  - Copy-on-write: Enables sharing of files
- Applications: Sandboxing, Fine grain access control (root in the container)

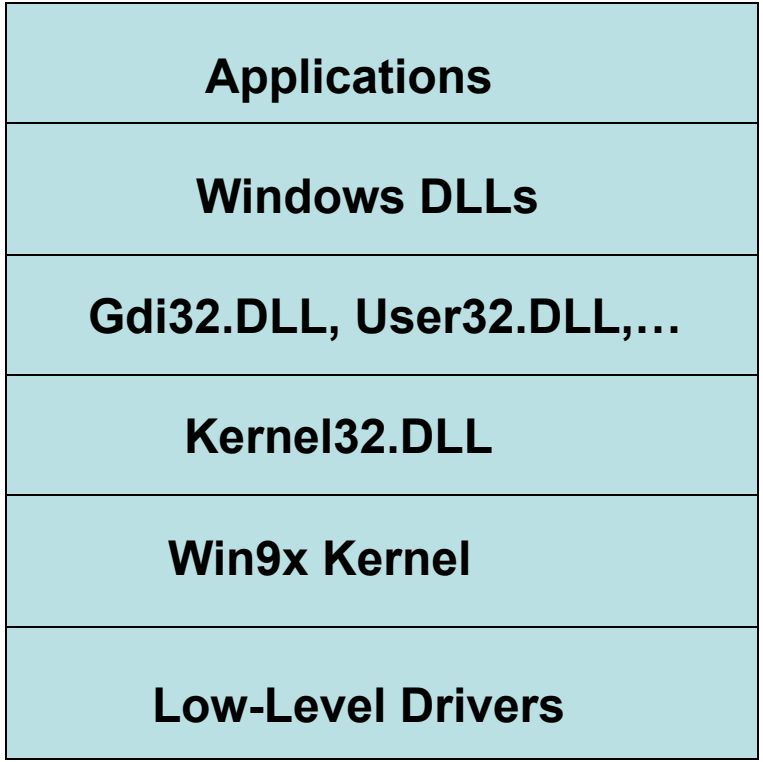
OS Level Virtualization: Examples

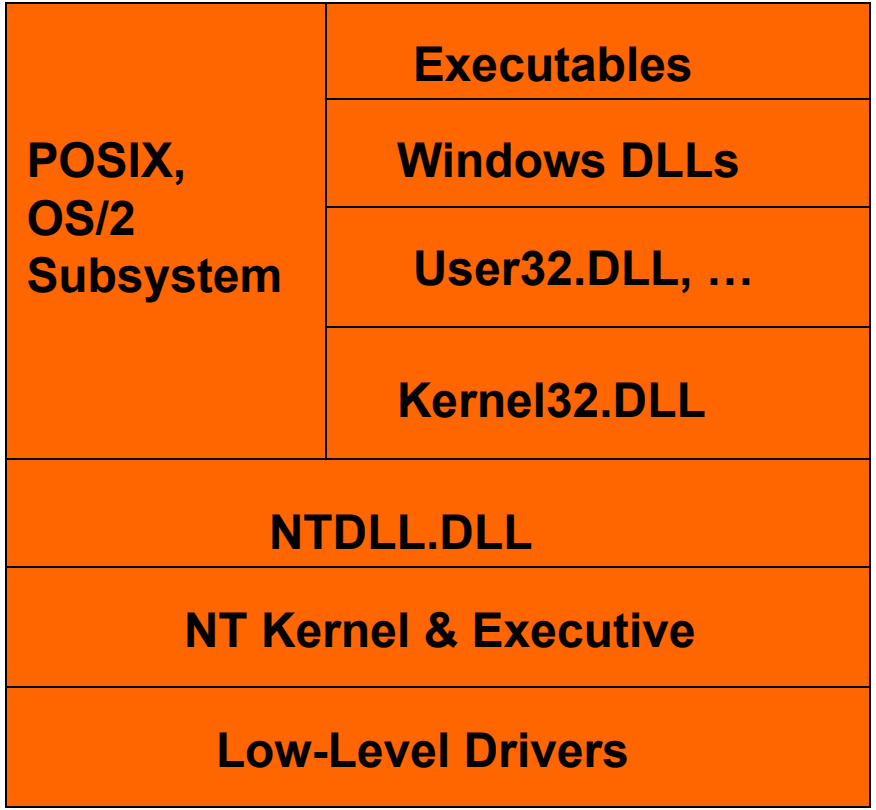
- Jail
  - FreeBSD based virtualization using *"chroot()"*
  - Scope is limited to the *jail*
  - Curtailed access to resources and operations
    - Signals, debugger, IP spoofing, system calls
  - A file-system sub-tree, one IP address, one "root"
- Ensim's "Virtual Private Server"
  - Supports virtual "boot", per-VM resource limits
  - Virtual /proc, IP address-space
- Linux "Virtual Environment" (VE)
  - Tagged VE (VE-id), policy support for the rights of "root"

Library Level Virtualization

- Technologies
  - API interception through DLL hooking
  - Partial/complete implementation of APIs
  - Emulate low level kernel implementations in user-space
    - Useful when the host OS does not provide required support (e.g. Win32 threads vs. pthreads)
    - Mandatory drivers
- Examples
  - WINE: Win32 API implementation on Unix/X
  - POSIX, OS/2 subsystems on Windows
    - Supports Unix and OS/2 like API
  - LxRun: Linux API implementation on SCO UnixWare, Solaris
  - WABI: Sun's implementation similar to WINE (not extensive)

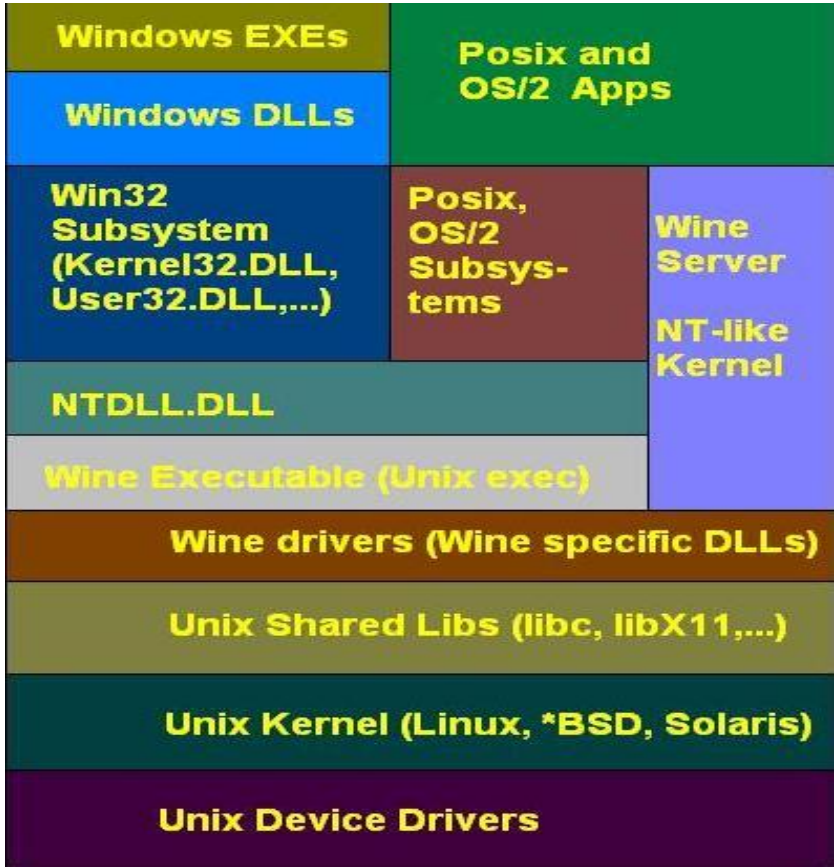
Windows Architecture





Wine Architecture

- Closely follows NT
  - Implements all the "core" DLLs (ntdll, user32, kernel32)
- Wine server provides the NT backbone
  - Message passing
  - Synchronization
  - Object handles
- Native DLL support for non-core libraries
- Hardware access through



WINE Implementation

- Wine server
  - IPC through Unix sockets and shared message queues
  - Process/Thread management
  - Simulates Synchronization primitives
- Native vs. Built-in DLLs
  - DLLs are implemented as Unix shared libraries (built-in DLLs)
  - Supports non-core Windows DLLs (Native DLLs)
  - A fully implemented built-in DLL takes precedence over native DLLs
- Executable Load
  - DLL descriptors table maintain the list of loaded DLLs
  - Imports are resolved using DLL descriptor table or on-disk DLLs
- Processes/Threads
  - Windows processes are mapped to WINE/UNIX processes
  - Thread-related APIs implemented in user-space and using pthreads

Application Level Virtualization

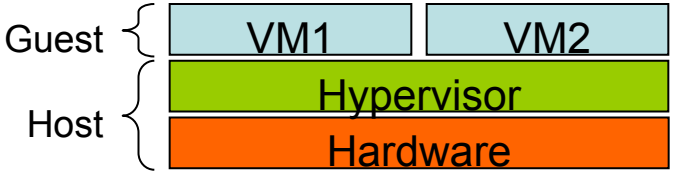
- Java Virtual Machine (JVM)
  - Executes Java byte code (virtual instructions)
  - Provides the implementation for the instruction set interpreter (or JIT compiler)
  - Provides code verification, SEH, garbage collection
  - Hardware access through underlying OS
- JVM Architecture
  - Stack-based architecture
  - No MMU
  - Virtual hardware: PC, register-set, heap, method (code) areas
  - Rich instruction set
    - Direct object manipulation, type conversion, exception throws
- Provides a runtime environment through JRE
- Other Examples: .NET CLI, Parrot (PERL 6)

Type	Description	Advantages	Disadvantages
Emulation	The hypervisor presents a complete virtual machine (of a foreign computing architecture to the host) enabling foreign applications to run in the emulated environment.	Simulates hardware that is not physically available.	Low performance and low density
Full	The hypervisor provides a complete virtual machine (of the same computing architecture as the host) enabling unmodified guests to run isolation.	Flexibility-run different versions of different operating systems from multiple vendors.	Guest OS does not know that it is being virtualized. Can incur a sizable performance hit on commodity hardware, particularly for I/O intensive applications.
Para	The hypervisor provides a complete but specialized virtual machine (of the same computing architecture as the host) to each guest allowing modified guests to run in isolation.	Lightweight and fast, near native speeds: Demonstrated to operate in the 0.5%-3.0% overhead range. [http://www.cl.cam.ac.uk/research/srg/netos/papers/2003-xen sosp.pdf] Allows OS to cooperate with hypervisor – improves IO and resource scheduling. Allows virtualizing architecture that do not support full virtualization.	Requires porting quest OS to use hypercalls instead of sensitive instructions. The main limitataion of paravirtualization is the guest OS must be tailored specifically to run on top of the virtual machine monitor(VMM), the host program that support multiple, identical execution environments. This especially impacts legacy closed source operating systems that have not yet implemented paravirtualized extensions.
OS level	A single operating system is modified in such a way as to allow various user space server processes to be coalesced into functional unites, which are executed in isolation from one another while running on a single hardware platform.	Fast, lightweight virtualization layer. It has the best possible (that is, close to native) performance and density, and features dynamic resource and management.	In practice, strong isolation is difficult to implement. Requires the same OS and patch level on all virtualized machines (homogeneous computing infrastructure).
Library	Emulates operating systems or subsystems via a special software library. Does not provide the illusion of a stand-alone system with a full operating system.	Provides missing API for application developers.	Often performs more slowly than a native optimized port of the application.
Application	Applications run in a virtual execution environment that provides a standard API for cross-platform execution and manages the application's consumption of local resources.	Manages resources automatically, which eases programmer learning curve. Increase portability of applications.	Execution is slower than native code. Overhead of virtual machine incurred when compared to native code.

Third Classification: Two types of hypervisors

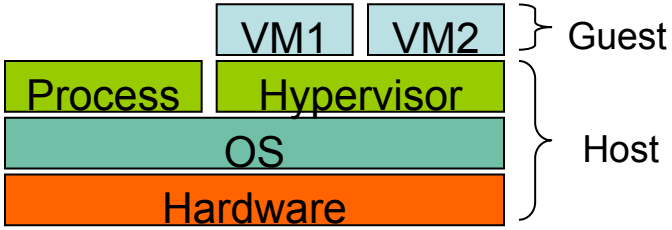
- Definitions
  - **Hypervisor** (or **VMM** – Virtual Machine Monitor) is a software layer that allows several **virtual machines** to **run** on a **physical machine**
  - The physical OS and hardware are called the **Host**
  - The virtual machine OS and applications are called the **Guest**

Type 1 (bare-metal)



VMware ESX, Microsoft Hyper-V, Xen

Type 2 (hosted)



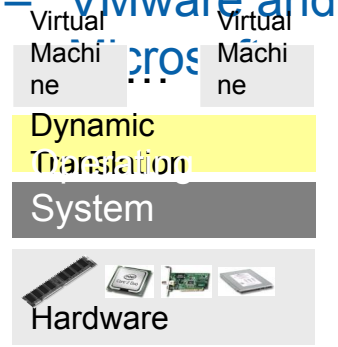
VMware Workstation, Microsoft Virtual PC, Sun VirtualBox, QEMU, KVM

Bare-metal or hosted?

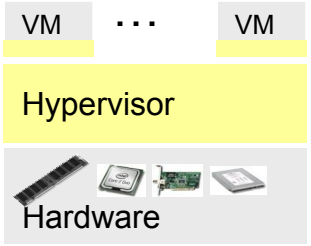
- **Bare-metal**
  - Has complete **control over hardware**
  - Does not have to **"fight" an OS**
- **Hosted**
  - Avoid **code duplication**: need not code a **process scheduler**, **memory management** system – the **OS already does** that
  - Can run native **processes alongside** VMs
  - Familiar environment – **how much CPU** and **memory** does a VM take? Use **top**! How big is the **virtual disk**? **ls –l**
  - Easy management – stop a VM? Sure, just kill it!
- A combination
  - Mostly hosted, but some parts are inside the OS kernel for performance reasons
  - E.g., **KVM**

Evolution of Software solutions*

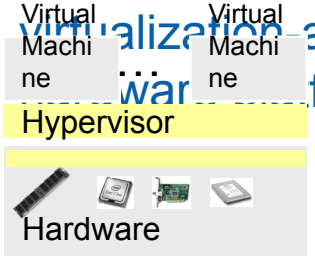
- 1st Generation: Full virtualization (Binary rewriting)
  - Software Based
  - VMware and Microsoft



- 2nd Generation: Paravirtualization
  - Cooperative virtualization
  - Modified guest
  - VMware, Xen



- 3rd Generation: Silicon-based (Hardware-assisted) virtualization
  - Unmodified guest
  - VMware and Xen on virtualization-aware hardware platforms



Time

Virtualization Logic

*This slide is from Intel® Corporation

I/O Virtualization

I/O Virtualization

- We saw **methods** to **virtualize** the **CPU**
- A computer is more than a CPU
- Also need I/O!

- Types of I/O:
  - Block (e.g., hard disk)
  - Network
  - Input (e.g., keyboard, mouse)
  - Sound
  - Video
- Most performance critical (for servers):
  - Network
  - Block

Side note – How does a NIC (network interface card) driver work?

- Transmit path:
  - OS prepares packet to transmit in a buffer in memory
  - Driver writes **start address** of buffer to **register X** of the NIC
  - Driver writes **length** of buffer to **register Y**
  - Driver writes '1' (**GO!**) into **register T**
  - NIC reads packet from memory addresses [X,X+Y) and sends it on the wire
  - NIC sends interrupt to host (**TX complete**, next packet please)
- Receive path:
  - Driver prepares buffer to receive packet into
  - Driver writes **start address** of buffer to **register X**
  - Driver writes **length** of buffer to **register Y**
  - Driver writes '1' (**READY-TO-RECEIVE**) into **register R**
  - When packet arrives, NIC copies it into memory at [X,X+Y)
  - NIC interrupts host (**RX**)
  - OS processes packet (e.g., wake the waiting process up)

I/O Virtualization? Emulate!

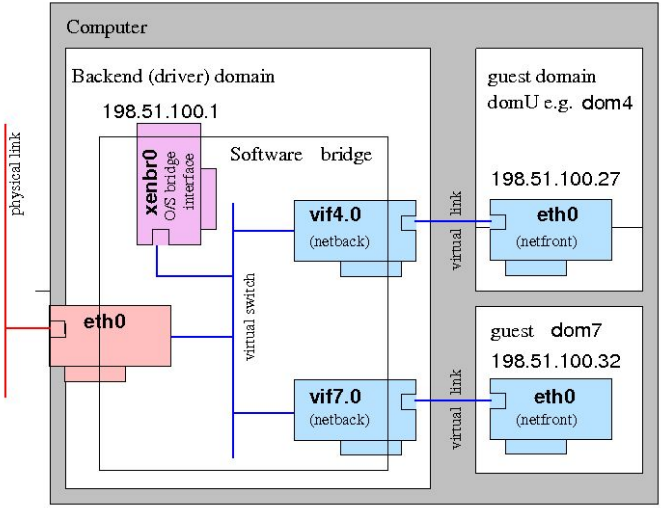
- Hypervisor implements **virtual NIC** (by the specification of a real NIC, e.g., Intel, Realtek, Broadcom)
- NIC **registers** (X, Y, Z, T, R, …) are just **variables** in hypervisor (host) **memory**
- If **guest writes** '1' to **register T**, **hypervisor reads** buffer from **memory [X,X+Y)** and passes it to **physical NIC** driver for transmission
- When physical NIC interrupts (**TX complete**), hypervisor **injects** TX complete interrupt into guest

- Similar for RX path

I/O Virtualization? Emulate!

- Pro:
  - **Unmodified guest** (guest already has drivers for Intel NICs…)
- Cons:
  - **Slow** – every access to every NIC register causes a **VM exit** (trap to hypervisor)
  - Hypervisor needs to **emulate complex hardware**
- Example hypervisors: QEMU, KVM, VMware (without VMware Tools)

I/O Virtualization? Paravirtualize!



- Add virtual NIC driver into guest (**frontend**)
- Implement the virtual NIC in the hypervisor (**backend**)
- Everything works just like in the emulation case…
- …except – **protocol** between frontend and backend
- Protocol in emulation case:
  - Guest writes registers X, Y, waits at least 3 nano-sec and writes to register T
  - Hypervisor **infers** guest wants to transmit packet
- Paravirtual protocol:
  - Guest does a hypercall, passes it start address and length as arguments
  - Hypervisor **knows** what it should do
- Paravirtual protocol can be **high-level**, e.g., ring of buffers to transmit (so NIC

I/O Virtualization? Paravirtualize!

- Pro:
  - **Fast** – no need to emulate physical device
- Con:
  - Requires **guest driver**
- Example hypervisors: QEMU, KVM, VMware (with VMware Tools), Xen
- How is paravirtual I/O different from paravirtual guest?
  - Paravirtual guest requires to modify **whole OS**
    - Try doing it on Windows (without source code), or even Linux (lots of changes)
  - Paravirtual I/O requires the addition of a **single driver** to a guest!

I/O Virtualization? Direct access / direct assignment!

- "Pull" NIC out of the host, and "plug" it into the guest
- Guest is allowed to access NIC registers **directly**, no hypervisor intervention
- Host can't access NIC anymore

I/O Virtualization? Direct access / direct assignment!

- Pro:
  - As **fast** as possible!

- Cons:
  - Need NIC per guest
  - Plus one for host
  - Can't do "cool stuff"
    - Encapsulate guest packets, monitor, modify them at the hypervisor level

I/O Virtualization? Emerging standard – SR-IOV!

- Single root I/O virtualization
- Contains a **physical function** controlled by the host, used to create **virtual functions**
- Each virtual function is assigned to a guest (like in **direct assignment**)
- Each **guest thinks** it has **full control** of NIC, accesses registers directly
- NIC does multiplexing/demultiplexing of traffic

I/O Virtualization? Emerging standard – SR-IOV!

- Pros:
  - As **fast** as possible!
  - Need only one NIC (as opposed to direct assignment)

- Cons:
  - Emerging standard
    - Few hypervisors fully support it
    - Expensive!
    - Requires new hardware
  - Can't do "cool stuff"

- Example hypervisors: KVM, Xen, VMware

Industry trends on I/O virtualization

- SR-IOV is the fastest, however it is the most expensive solution
- Paravirtual I/O is cheap, but much worse performance
- Companies (Red Hat, IBM, …) are looking at paravirtual I/O, trying to optimize it

Virtual Machine Monitor (VMM)

Concepts

References and Sources

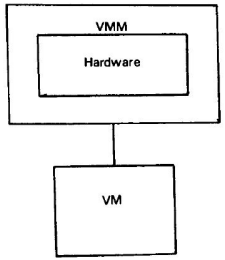
- James Smith, Ravi Nair, “The Architectures of Virtual Machines,” IEEE Computer, May 2005, pp. 32-38.
- Mendel Rosenblum, Tal Garfinkel, “Virtual Machine Monitors: Current Technology and Future Trends,” IEEE Computer, May 2005, pp. 39-47.
- L.H. Seawright, R.A. MacKinnon, “VM/370 – a study of multiplicity and usefulness,” IBM Systems Journal, vol. 18, no. 1, 1979, pp. 4-17.
- S.T. King, G.W. Dunlap, P.M. Chen, “Operating System Support for Virtual Machines,” Proceedings of the 2003 USENIX Technical Conference, June 9-14, 2003, San Antonio TX, pp. 71-84.
- A. Whitaker, R.S. Cox, M. Shaw, S.D. Gribble, “Rethinking the Design of Virtual Machine Monitors,” IEEE Computer, May 2005, pp. 57-62.
- G.J. Popek, and R.P. Goldberg, “Formal requirements for virtualizable third generation architectures,” CACM, vol. 17 no. 7, 1974, pp. 412-421.

Definitions

- Virtual Machine Monitor (VMM)
  - A virtualization system that partitions a single physical "machine" into multiple virtual machines.
  - Terminology
    - Host – the machine and/or software on which the VMM is implemented
    - Guest – the OS which executes under the control of the VMM

Origins - Principles

Fig. 1. The virtual machine monitor.



"an *efficient, isolated duplicate* of the real machine"

- Efficiency
  - Innocuous instructions should execute directly on the hardware
- Resource control
  - Executed programs may not affect the system resources
- Equivalence
  - The behavior of a program executing under the VMM should be the same as if the program were executed directly on the hardware (except possibly for timing and resource availability)

Formal Requirements for Virtualizable Third Generation Architectures

Gerald J. Popek
University of California, Los Angeles
and
Robert P. Goldberg
Honeywell Information Systems and
Harvard University

Virtual machine systems have been implemented on a limited number of third generation computer systems, e.g. CP-67 on the IBM 360/67. From previous empirical studies, it is known that certain third generation computer systems, e.g. the DEC PDP-10, cannot support a virtual machine system. In this paper, model of a third-generation-like computer system is developed. Formal techniques are used to derive precise sufficient conditions to test whether such an architecture can support virtual machines.

Communications of the ACM, vol 17, no 7, 1974, pp.412-421

Origins - Principles

Instruction types

- Privileged

  an instruction traps in unprivileged (user) mode but not in privileged (supervisor) mode.

- Sensitive
  - Control sensitive –

    attempts to change the memory allocation or privilege mode
  - Behavior sensitive
    - Location sensitive – execution behavior depends on location in memory
    - Mode sensitive – execution behavior depends on the privilege mode
- Innocuous – an instruction that is not sensitive

Theorem

For any conventional third generation computer, a virtual machine monitor may be constructed if the set of sensitive instructions for that computer is a subset of the set of privileged instructions.

Origins - Technology

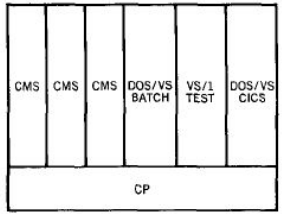
VM/370—a study of multiplicity and usefulness

by L. H. Seawright and R. A. MacKinnon

The productivity of data processing professionals and other professionals can be enhanced through the use of interactive and time-sharing systems. Similarly, system programmers can benefit from the use of system testing tools. A systems solution to both areas can be the virtual machine concept, which provides multiple software replicas of real computing systems on one real processor. Each virtual machine has a full complement of input/output devices and provides functions similar to those of a real machine. One system that implements virtual machines is IBM's Virtual Machine Facility/370 (VM/370).[1]

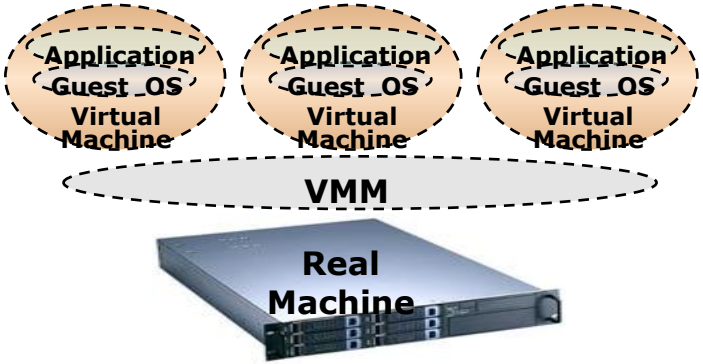
IBM Systems Journal, vol. 18, no. 1, 1979, pp. 4-17.

Figure 1 A VM/370 environment

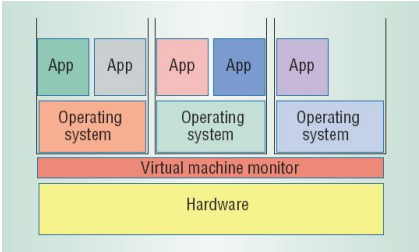


- Concurrent execution of multiple production operating systems
- Testing and development of experimental systems
- Adoption of new systems with continued use of legacy systems
- Ability to accommodate applications requiring special-purpose OS
- Introduced notions of "handshake" and "virtual-equals-real mode" to allow sharing of resource control information with CP
- Leveraged ability to co-design hardware, VMM, and guestOS

VMMs Rediscovered

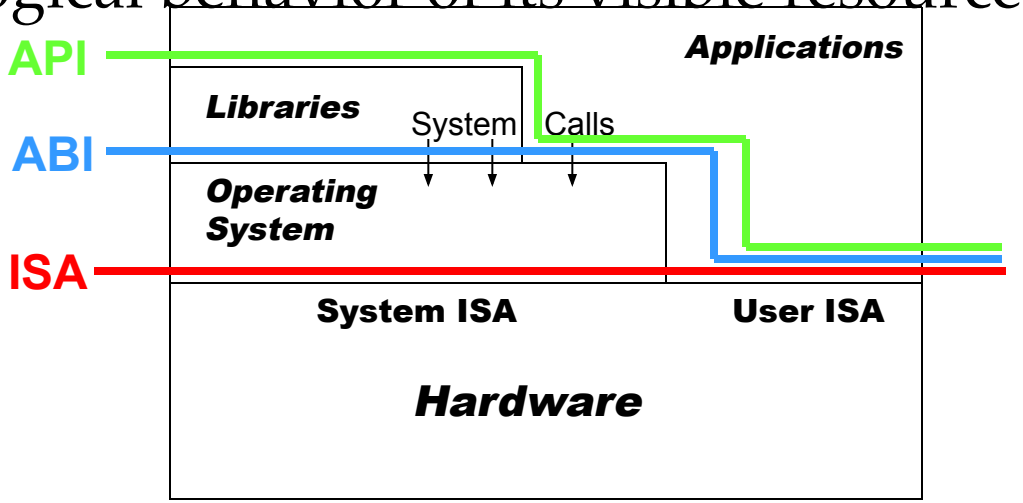


- Server/workload consolidation (reduces “server sprawl”)
- Compatible with evolving multi-core architectures
- Simplifies software distributions for complex environments
- “Whole system” (workload) migration
- Improved data-center management and efficiency
- Additional services (workload isolation) added “underneath” the OS
  - security (intrusion detection, sandboxing,…)



Architecture & Interfaces

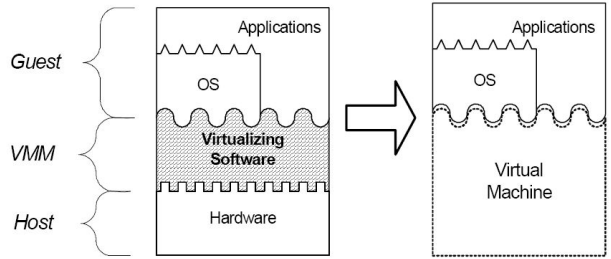
- Architecture: formal specification of a system's interface and the logical behavior of its visible resources.



- API – application binary interface
- ABI – application binary interface
- ISA – instruction set architecture

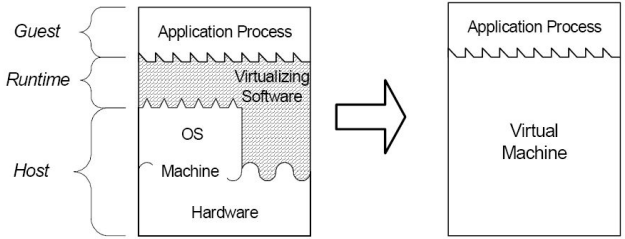
VMM Types

- System



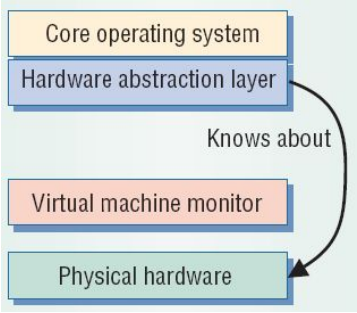
- Provides ABI interface
- Efficient execution
- Can add OS-independent services (e.g., migration, intrustion detection)

- Process

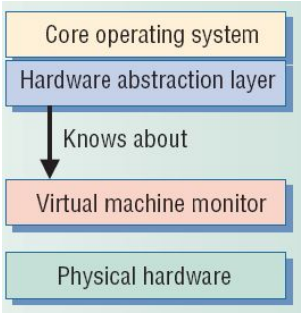


- Provides API interface
- Easier installation
- Leverage OS services (e.g., device drivers)
- Execution overhead (possibly mitigated by just-in-time compilation)

System-level Design Approaches

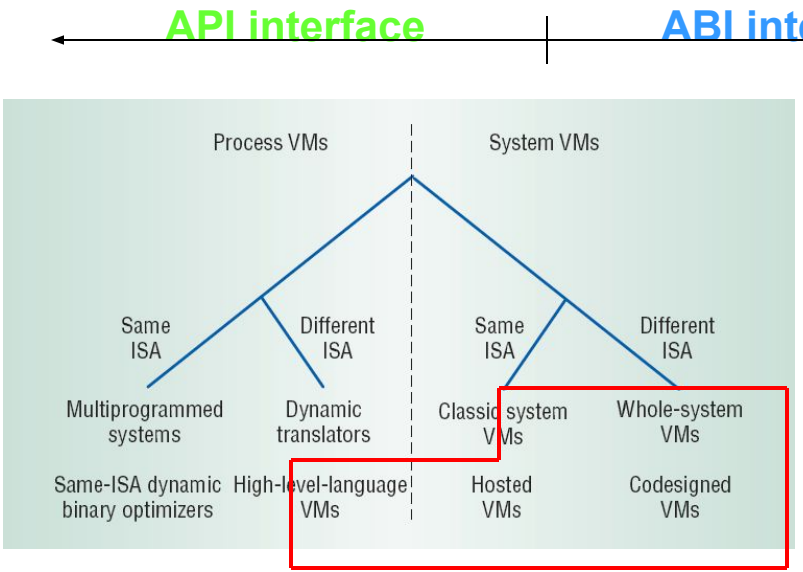


- Full virtualization (direct execution)
  - Exact hardware exposed to OS
  - Efficient execution
  - OS runs unchanged
  - Requires a "virtualizable" architecture
  - Example: VMWare



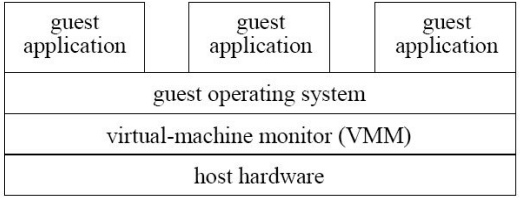
- Paravirtualization
  - OS modified to execute under VMM
  - Requires porting OS code
  - Execution overhead
  - Necessary for some (popular) architectures (e.g., x86)
  - Examples: Xen, Denali

Design Space (level vs. ISA)

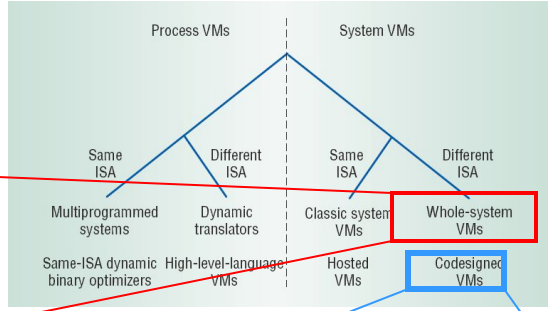


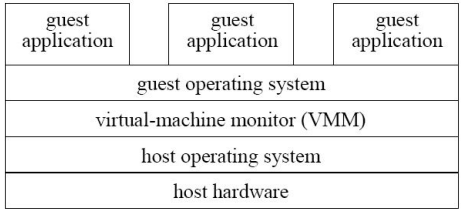
- Variety of techniques and approaches available
- Critical technology space highlighted

System VMMs



Type 1



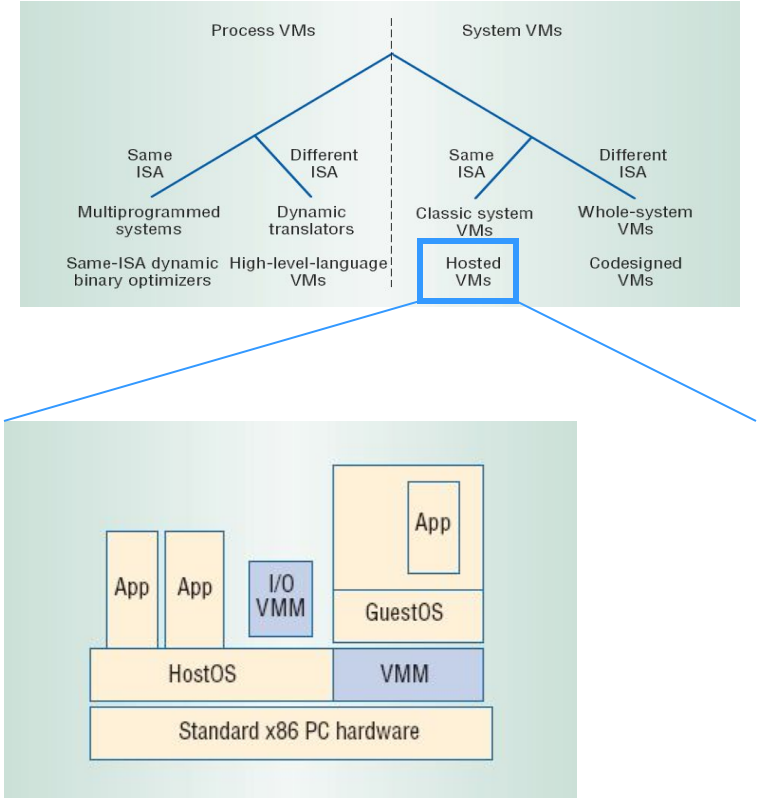


Type 2

- Structure
  - Type 1: runs directly on host hardware
  - Type 2: runs on HostOS
- Primary goals
  - Type 1: High performance
  - Type 2: Ease of construction/installation/acceptability
- Examples
  - Type 1: VMWare ESX Server, Xen, OS/370

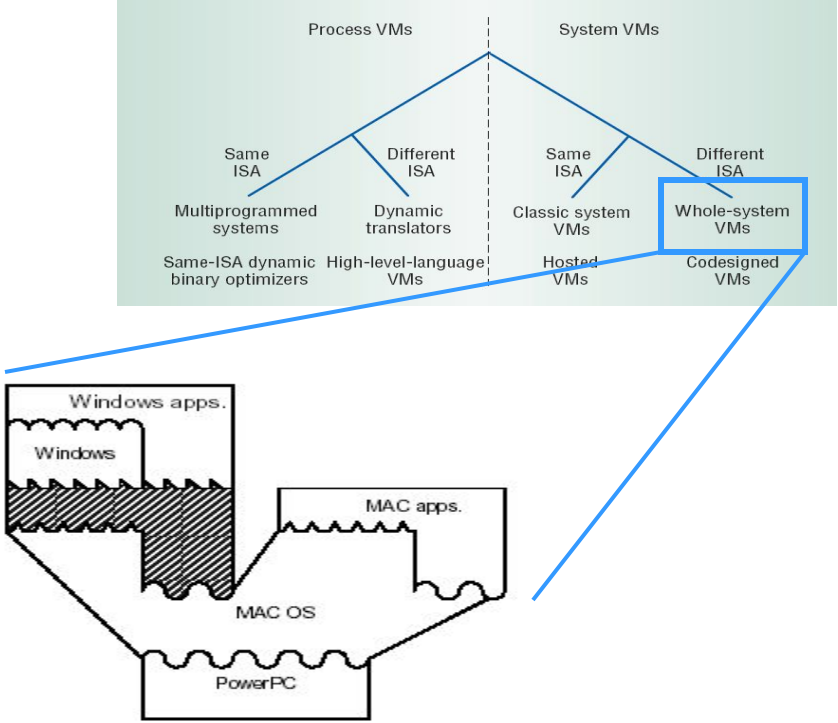
Hosted VMMs

- Structure
  - Hybrid between Type1 and Type2
  - Core VMM executes directly on hardware
  - I/O services provided by code running on HostOS
- Goals
  - Improve performance overall
  - leverages I/O device support on the HostOS
- Disadvantages
  - Incurs overhead on I/O operations
  - Lacks performance isolation and performance guarantees

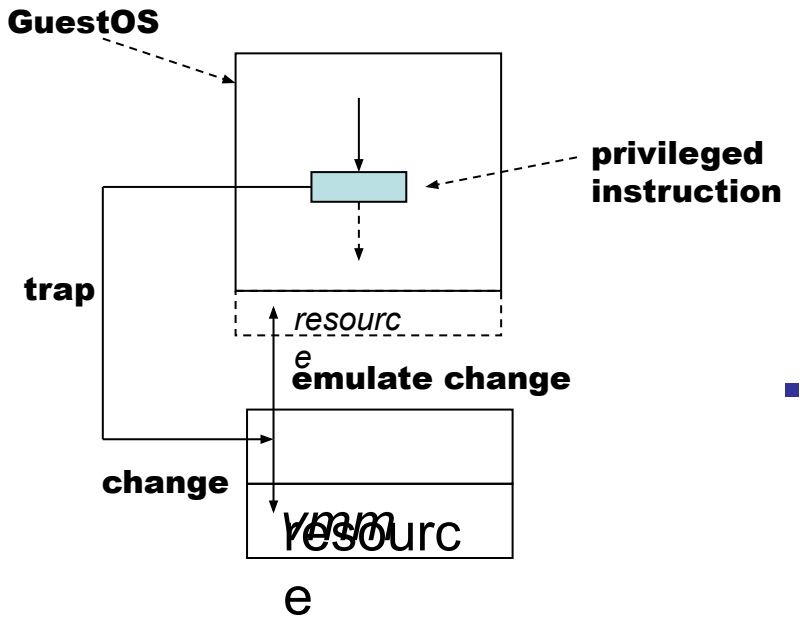


Whole-system VMMs

- Challenge: GuestOS ISA differs from HostOS ISA
- Requires full emulation of GuestOS and its applications
- Example: VirtualPC

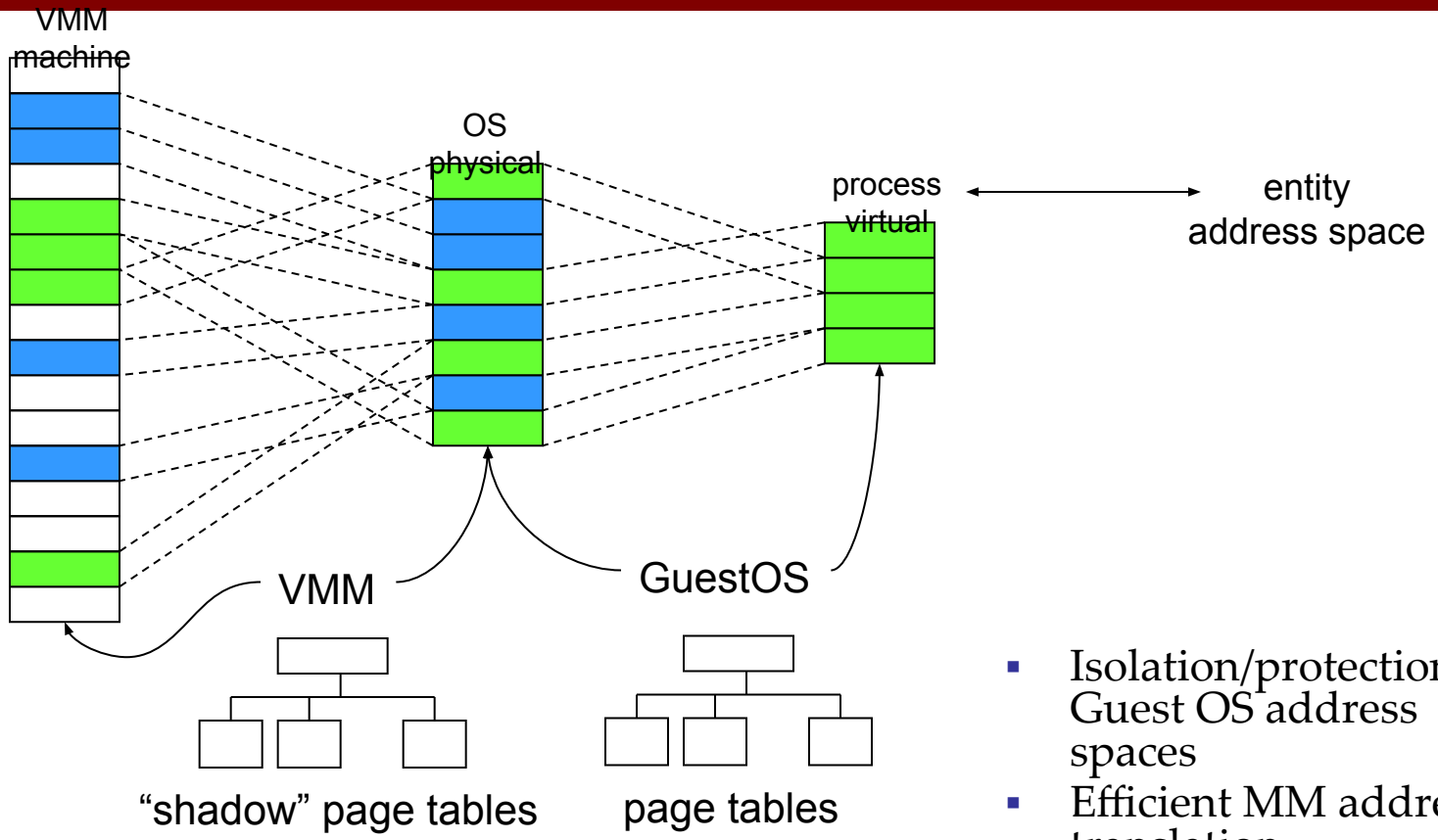


Strategies



- De-privileging
  - VMM emulates the effect on system/hardware resources of privileged instructions whose execution traps into the VMM
  - aka trap-and-emulate
  - Typically achieved by running GuestOS at a lower hardware priority level than the VMM
  - Problematic on some architectures where privileged instructions do not trap when executed at deprivileged priority
- Primary/shadow structures
  - VMM maintains “shadow” copies of critical structures whose “primary” versions are manipulated by the GuestOS
  - e.g., page tables
  - Primary copies needed to insure correct environment visible to GuestOS
- Memory traces

Memory Management



- Isolation/protection of Guest OS address spaces
- Efficient MM address translation

Clusters

Portions of this PPT draw from PPT authored by ShopniL Mahmud

Table of Contents

- Introducing Cluster Concept
- About Cluster Computing
- Concept of whole computers and it's benefits
- Architecture and Clustering Methods
  Different clusters catagorizations
  Issues to be consitered about clusters
- Implementations of clusters
  Clusters technology in present and future
  Conclusions

Introducing Clusters Computing

- A computer cluster is a group of tightly coupled computers that work together closely so that it can be viewed as a single computer.
- Clusters are commonly connected through fast local area networks.
- Clusters have evolved to support applications ranging from e-commerce, to high performance database applications.

Cluster Computers in view

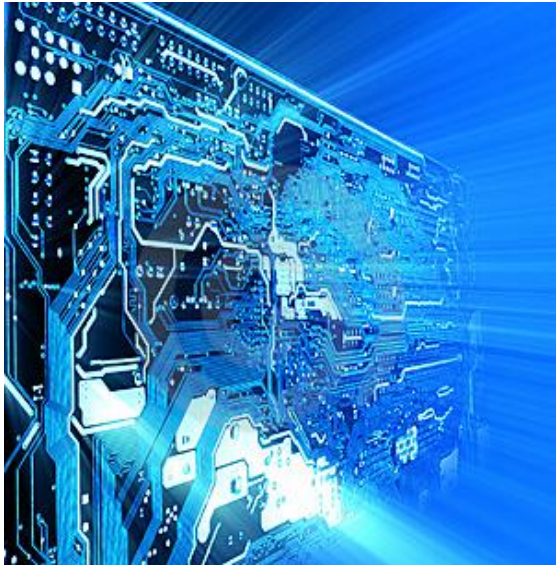
Cluster Computing

A group of interconnected <u>WHOLE COMPUTERS</u> works together as a unified computing resource that can create the illusion of being one machine having parallel processing.

The components of a cluster are commonly, but not always, connected to each other through fast local area networks.

What's Whole Computer

- A system that can refer run on its own apart from the cluster; used in server systems are called whole computers.



Why is Clusters than single 1's?

- Price/Performance

  The reason for the growth in use of clusters is that they have significantly reduced the cost of processing power.

- Availability

  Single points of failure can be eliminated, if any one system component goes down, the system as a whole stay highly available.

- Scalability

  HPC clusters can grow in overall capacity because processors and nodes can be added as demand increases.

Where does it matter?

- The components critical to the development of low cost clusters are:
  - Processors
  - Memory
  - Networking components
  - Motherboards, busses, and other sub-systems

Short History …

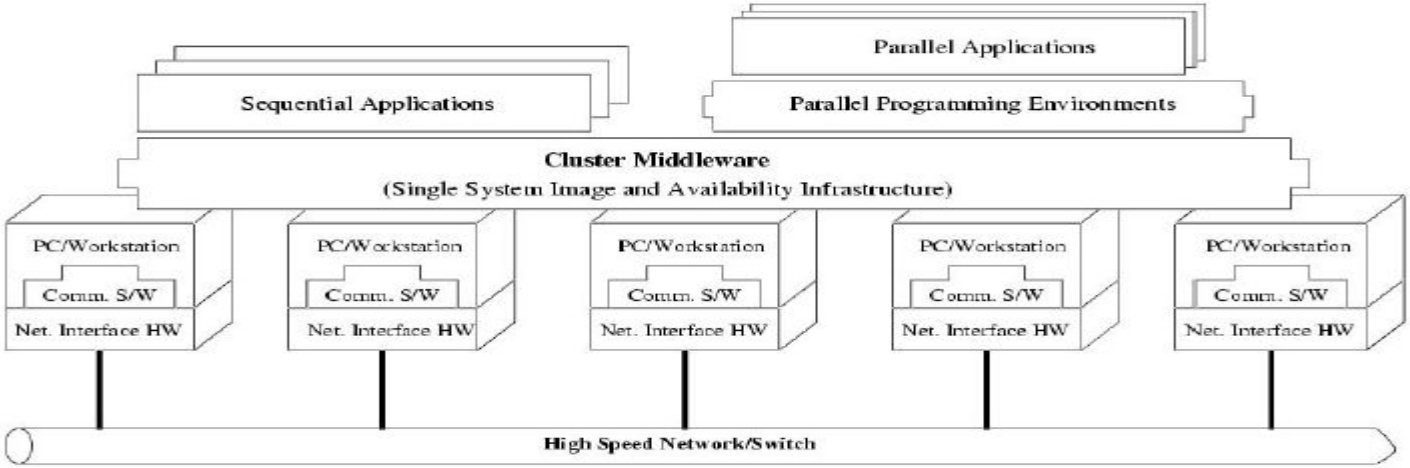
- The first commodity clustering product was ARCnet, developed by Datapoint in 1977.
- The next product was VAXcluster, released by DEC in 1980's.
- Microsoft, Sun Microsystems, and other leading hardware and software companies offer clustering packages.

IBM hidro Clusters

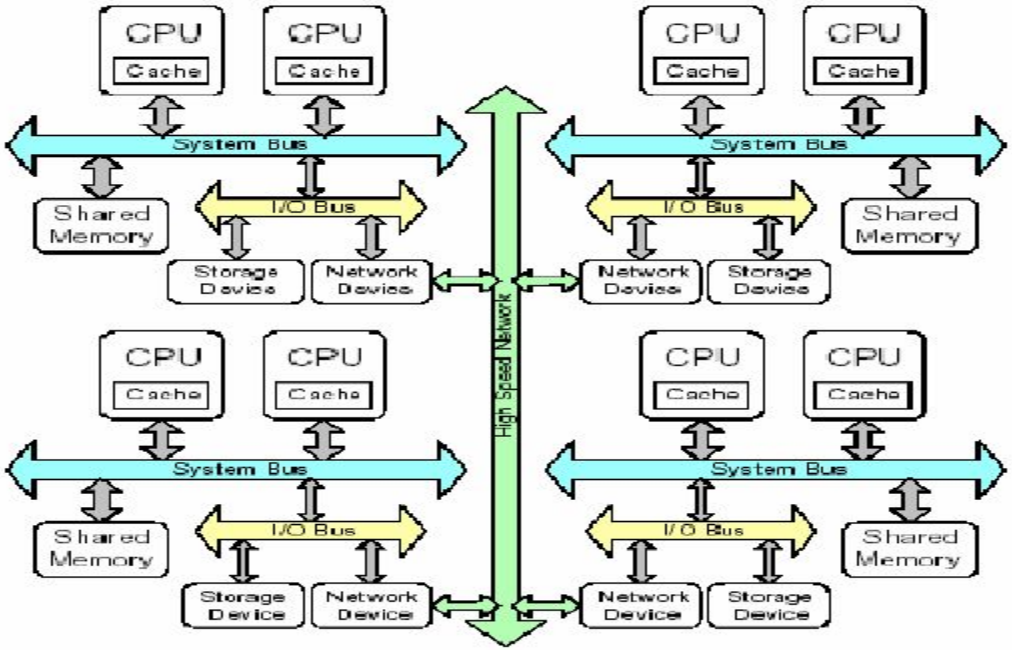
Clusters Architecture

- A cluster is a type of parallel /distributed processing system,which consists of a collection of interconnected stand-alone computers cooperatively working together a single, integrated computing resource.
- A node:
  - a single or multiprocessor system with memory, I/O facilities, &OS
  - generally 2 or more computers (nodes) connected together
  - in a single cabinet, or physically separated & connected via a LAN

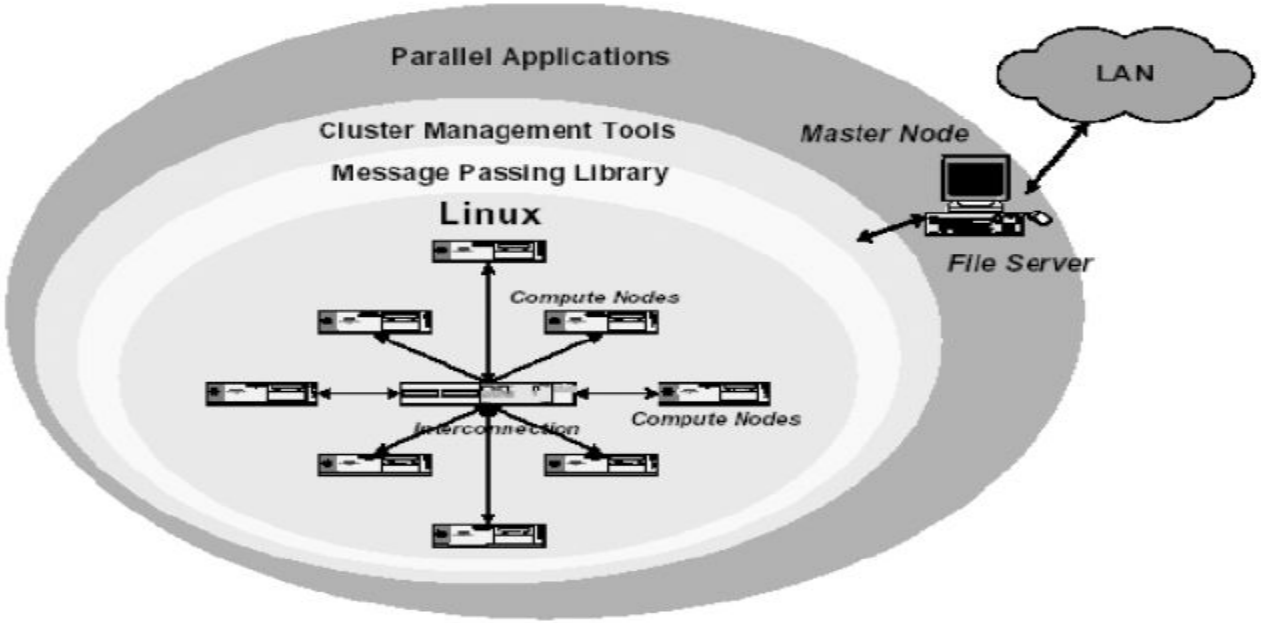
Architecture of Clusters



How Clusters works?

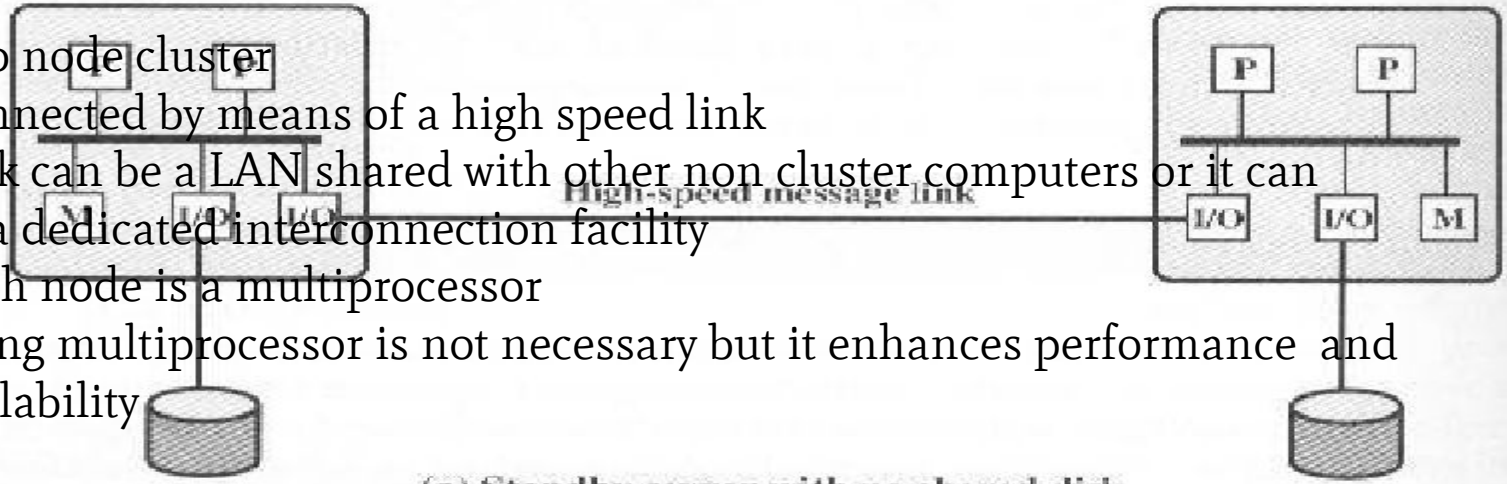


A logical view for clusters



Configuration Of Figure A

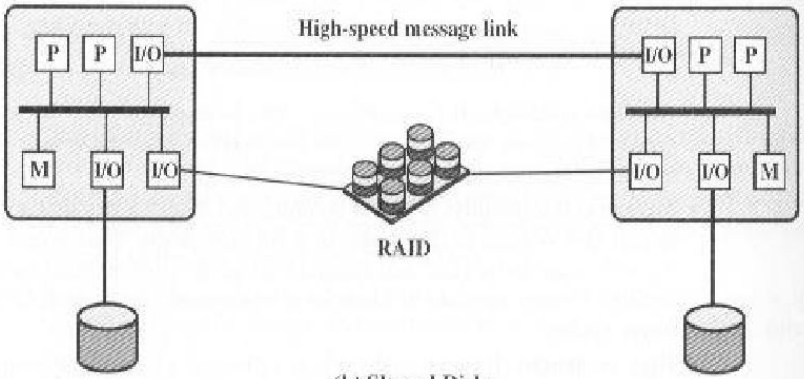
- Two node cluster
- Connected by means of a high speed link
- Link can be a LAN shared with other non cluster computers or it can
- be a dedicated interconnection facility
- Each node is a multiprocessor
- Being multiprocessor is not necessary but it enhances performance and availability



(a) Standby server with no shared disk

Configuration of figure B

Shared disk cluster
Message link between nodes
Also, there is a disk subsystem directly linked to multiple computers within the cluster
Common disk subsystem is a RAID
RAID is used so that high availability is not compromised by a shared disk that is a single point of failu



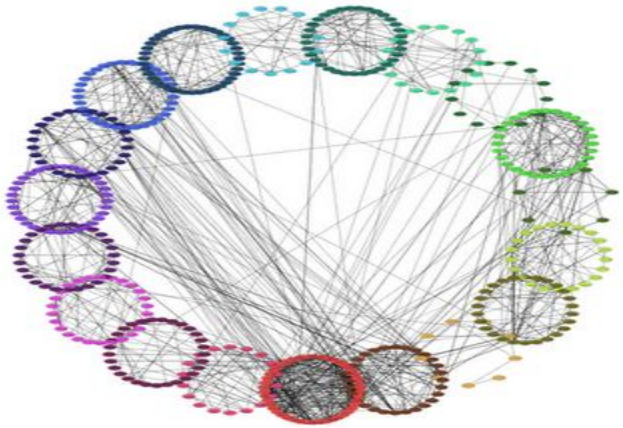
(b) Shared Disk

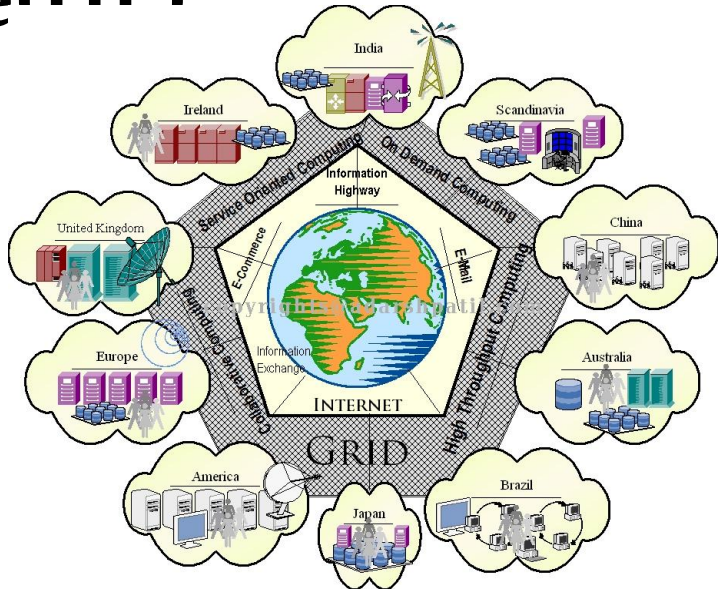
Clustering Methods

CLUSTERING METHOD	DESCRIPTION	BENEFITS	LIMITATIONS
Passive standby	A secondary server takes over in case of primary server failure	Easy to implement	High cost because the secondary server is unavailable for other processing tasks
Active standby	The secondary server is also used for processing tasks	Reduced cost because secondary servers can be used for processing	Increased complexity
Separate servers	Separate servers have their own disks. Data are continuously copied from primary to secondary server	High availability	High network and server overhead due to copying operations
Servers connected to disks	Servers are cabled to the same disks, but each server owns its disk. If one server fails, its disks are taken over by the other server	Reduced network and server overhead due to elimination of copying operations	Usually requires disk mirroring or RAID technology to compensate for risk of disk failure
Servers share disks	Multiple servers simultaneously share access to the disks	Low network and server overhead. Reduced risk of downtime caused by disk failure	Requires lock manager software. Usually used with disk mirroring or RAID technology

Cluster Catagorization

- **High-availability (HA)**
- **Load-balancing**
- **High- Performance(HP)**



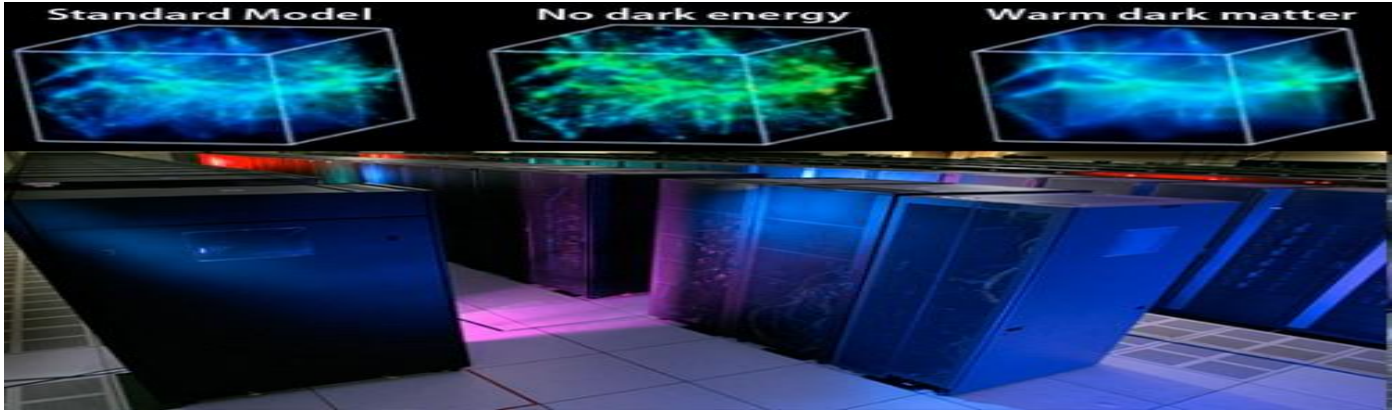


High Availability Clusters

- Avoid single point of failure
- This requires atleast two nodes - a primary and a backup.
- Always with redundancy
- Almost all load balancing cluster are with HA capability.

Load Balancing Clusters

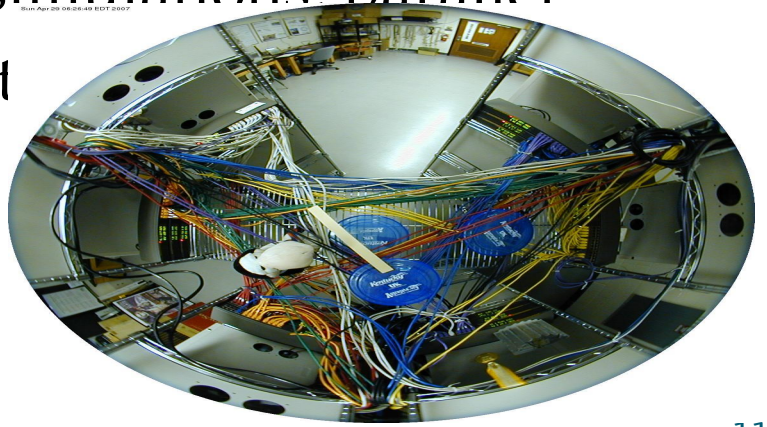
- PC cluster deliver load balancing performance
- Commonly used with busy ftp and web servers with large client base
- Large number of nodes to share load



High Performance Clusters

- Start from 1994
- Donald Becker of NASA assembled this cluster.
- Also called Beowulf cluster
- Applications like data mining, simulations, parallel

modeling, et



Issues to be considered about

- Cluster Networking
- Cluster Software
- Programming
- Timing
- Network Selection
- Speed Selection

Cluster Networking

- Huge difference in the speed of data accessibility and transferability and how the nodes communicate.

- Just got to make sure that if it's in your budget then the clusters have the similar networking capabilities and if possible, then b twork from e manufacturer.

Cluster Software

- You will have to build versions of clustering software for each kind of system you include in your cluster.

Programming

- Our code will have to be written to support the lowest common denominator for data types supported by the least powerful node in our cluster. With mixed machines, the more powerful machines will have attributes that cannot be attained in the powerful machine

TiminG

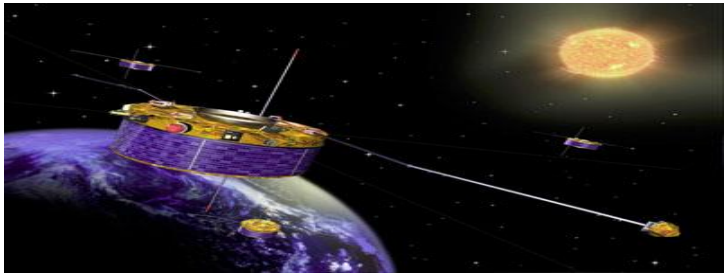
- Timing

  This is the most problematic aspect of cluster. Since these machines have different performance profile our code will execute at different rates on the different kinds of nodes. This can cause serious bottlenecks if a process on one node is waiting for results of a calculation on a slower node..

Network Selection

- Network Selection

  There are a number of different kinds of network topologies, including buses, cubes of various degrees, and grids/meshes. These network topologies will be implemented by use of one or more network interface cards, or NICs, installed into the head-node and compute nodes of our cluster.



Right Speed Selection

- Speed Selection

  No matter what topology you choose for your cluster, you will want to get fastest network that your budget allows. Fortunately, the availability of high speed computers has also forced the development of high speed networking systems.

- Examples are :

  10Mbit Ethernet, 100Mbit Ethernet, gigabit networking, channel bonding etc.

Implementation of Clusters

- The TOP 500 organization's semi-annual list of the 500 fastest computers usually includes many clusters.

- As of June 18, 2008, the top supercomputer is the Department of Energy's IBM Roadrunner system with performance of 1026 TFlops measured with High-Performance LINPACK benchmark.

- Clustering can provide significant performance

Implementation of Clusters

- the 28th most powerful supercomputer on Earth as of June 2006, is a 12.25 TFlops computer cluster of 1100 Apple XServe G5 2.3 GHz dual-processor machines (4 GB RAM, 80 GB SATA HD) running Mac OS X and using InfiniBand interconnect.

- The total cost of the previous Power Mac system was $5.2 million, a tenth of the cost of slower mainframe computer supercomputers. (The Power Mac G5s were sold off.)

- The central concept of a Beowulf cluster is the use of commercial off-the-shelf (COTS) computers to produce a cost-effective alternative to a traditional

Implementation of Clusters

- clusters are excellent for parallel computation, but much poorer than traditional supercomputers at non-parallel computation.
- JavaSpaces is a specification from Sun Microsystems that enables clustering computers via a distributed shared memory.
- gridMathematica - computer algebra and 3D visualization.
- High powered Gaming.

Cluster Technologies

- MPI is a widely-available communications library that enables parallel programs to be written in C, Fortran, Python, OCaml, and many other programming languages.
- The GNU/Linux world supports various cluster software; for application clustering and etc.
- Microsoft Windows Compute Cluster Server 2003 based on the Windows Server platform provides pieces for High Performance

Conclusion …

- Clusters are promising
- Solve parallel processing paradox
- New trends in hardware and software technologies are likely to make clusters.
- Clusters based supercomputers (Linux based clusters) can be seen everywhere !!!